

Understanding Machine Learning: The Mathematical Structure of Least-Squares Support Vector Machines

James Mark Weatherill

Supervisor: Dr Werner Bauer

Department of Mathematics

University of Surrey

Guildford GU2 7XH, United Kingdom

Submitted as a report for the B.Sc. Project for the B.Sc. degree
at the University of Surrey

May, 2026

Abstract

This dissertation aims to provide a rigorous mathematical treatment of least-squares support vector machines (LS-SVMs) starting from linear regression as the foundation. By working through the mathematics, we build up through kernel methods, LS-SVMs, and finally weighted LS-SVMs (wLS-SVMs). The report culminates in the derivation of the radial basis function (RBF) kernel for the wLS-SVM, and the evaluation of this model when applied to the modified National Institute of Standards and Technology (MNIST) dataset of handwritten digits. Our model achieves a peak accuracy of 98.76% on the testing set, after being trained on the entire training set of 60,000 points. On smaller subsets of the training data, our model achieves: 93.41% accuracy for 1,000 training points, 94.69% accuracy for 5,000 training points, and 98.00% accuracy for 15,000 training points. These results show that our wLS-SVM model is highly effective at generalising well to unseen data, even when trained on a small subset of the training data.

Keywords:

Support Vector Machines, Least-Squares Support Vector Machines, Weighted Least-Squares Support Vector Machines, Kernel Methods, Radial Basis Function Kernel, MNIST, Multi-Class Classification, Machine Learning

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Werner Bauer for giving me the opportunity and freedom to explore this topic in depth. His guidance, support, and encouragement have been invaluable throughout the course of this project.

Next, I would like to thank my family and friends for always supporting me and believing in me. In particular, I would like to thank my mother, father, sister and grandmother for their unwavering support and emotional encouragement, since their love and encouragement have been a source of motivation and strength for me.

I would also like to thank the wider academic community for the contributions they have made to the field of machine learning and support vector machines. Importantly, I would like to thank the authors of the many papers and books that I have read and cited in this report, with thanks to Vladimir Vapnik for his pioneering work on support vector machines, and Johan Suykens for his work on least-squares support vector machines.

Finally, I would like to thank the University of Surrey for providing me with the resources and support through the course of my studies, and for creating an environment that fosters learning and growth.

Contents

1. Introduction	1
2. Linear Regression	5
2.1. Why start with linear regression?	5
2.2. Creating our notation	6
3. Kernel Ridge Regression	9
3.1. Non-linear data	9
3.2. Keeping $\mathbf{X}$ manageable	11
3.3. The kernel	15
3.4. Reflection, and what's next?	18
4. Support Vector Machine History	19
4.1. Kernels across two centuries	19
4.2. The birth of support vector machines	21
4.3. Support vector machines meet the kernel	24
4.4. The least squares support vector machine	26
5. Least Squares SVMs	29
5.1. Derivation of the LS-SVM	29
5.2. Applying the LS-SVM to real data	36
5.3. The weighted LS-SVM	40
6. wLS-SVM model for MNIST dataset	47
6.1. Creating our infinite degree polynomial kernel	47
6.2. Multi-class classification	53
6.3. Multi-class RBF wLS-SVM results for MNIST dataset	56
7. Conclusion	61
A. Appendices	65
References	75

Introduction

Machine learning (ML) and artificial intelligence (AI) models have become increasingly complex and opaque in recent years, despite (or perhaps because of) their sophistication in digesting inputs and producing intelligible outputs. Due to this, the terms ‘Explainable AI’ (XAI) and ‘Interpretable ML’ (IML) have emerged among researchers and the wider public, as a response to the opacity of some of these models [1]. An example of these models gaining significant proficiency over a short period, would be image-recognition models. One study found that ChatGPT-4V (a text and image model) and ChatGPT-4o (also a text and image model) scored 71.0% and 77.1% accuracy respectively on a ‘Diagnose This’-style task [2]. The models were released less than one year apart (September 2023 and May 2024 respectively) and yet had more than a 6% increase in accuracy [3].

Mathematics is not just a tool that can be used to understand how machine learning models work, but it is the foundation on which all ML models are built. For many models, the underlying mathematics is difficult to decipher straight away, even for people with a strong mathematical background. This report aims to build up from the knowledge of a familiar mathematical concept, to an advanced ML model capable of recognising handwritten digits, while explaining all of the rigorous mathematics at each stage. Through this approach, sophisticated ML models can be understood in a more logical way.

This report will be structured in a pedagogical manner, starting with linear regression, and directly through to kernel ridge regression (KRR). It should be noted that regression, although not directly relevant to support vector machines (SVMs), will serve as a very useful stepping stone by helping understand the structure of the kernel. Understanding the mathematics behind support vector machines is the main goal, and the report will culminate in showing the mathematics of an infinite degree kernel (radial basis function kernel) for an SVM, and showing the results of an SVM model that is trained on the MNIST dataset of handwritten digits [4]. The specific SVM solving method that will be used is called a least squares support vector machine (LS-SVM), which will be explained in due course.

The reason behind choosing SVMs as the goal is due to their mathematical elegance, ability to be geometrically visualised, and their extreme effectiveness in classification tasks. SVMs sit nicely on the boundary between ‘white-box’ (easily understandable) models, like linear regression, and ‘black-box’ (more challenging) models, like the neural networks behind large language models (LLMs) [1]. This gives SVMs a uniquely powerful, but interpretable, position in the world of machine learning. SVMs are also still widely used, particularly for e-mail spam detection, as well as medical diagnosis and classification [5,6].

Ultimately, this report aims to shift the perception of machine learning being inherently opaque, towards something more concrete and valuable: mathematical understanding. By working through the rigorous mathematics, readers will gain not just knowledge of LS-SVMs, but also a foundational understanding of the underlying algorithms. This should give readers a higher level of confidence and trust in the models they use, as well as acting as a launchpad for further study into more complex models in the future.

The mathematical structure of this report draws from a broad body of literature, across regression, kernel methods, support vector machines. Derivations and explanations are original in their presentation, and where results are well-known, the treatment of the mathematics is original. Where appropriate, references to the literature are made, and contained in the bibliography.

Chapter 2 will cover the ‘why’ behind starting with linear regression, and we will cover well-established mathematical representation of linear regression formulae. **Chapter 3** will be devoted to introducing regression in higher degrees, utilising the kernel trick. This chapter lays down the most fundamental groundwork for understanding the higher-dimensional capabilities of support vector machines. Data and presentation of the results are the author’s own, while terminology and relevant mathematical results are cited from the literature.

Chapter 4 will cover the history of support vector machines, and how support vector machines came to be. It stems from kernels to least-squares support vector machines, and all relevant historical and mathematical results are cited from the literature. **Chapter 5** and **Chapter 6** return to the practical derivation and application of the weighted least-squares support vector machine, reaching the final model that is capable of classifying the 10,000 testing points from the MNIST dataset. Similar to previous chapters, all data is the author’s own, and application of the models to these datasets is done mathematically and practically by the author. Where specific results are used or brought up, they are cited from the literature.

Linear Regression

2.1. Why start with linear regression?

Linear regression is one of the most fundamental machine learning models that exists [7]. It is what is known as a ‘supervised learning’ model [7], which means that our data consists of inputs and outputs, and the model learns to predict new outputs, given new inputs, based on the data it has been trained on. This is the same type of learning we will be using for every model in this report. The alternative to supervised learning is, predictably, ‘unsupervised learning’, where our data consists of inputs only, meaning the model has to find patterns in the data without any guidance, and data is grouped into clusters based on these patterns [7]. Unsupervised learning is inherently more mathematically opaque than supervised learning, and would be impractical as a starting point when discussing XAI and IML.

Another, more important, reason for starting with linear regression is that it is the foundation for our goal model of an LS-SVM. Each part of the process of building up to an LS-SVM can be understood as small additions and/or modifications to the linear regression model.

Linear regression is something that all degree-level, or higher, mathematicians will be familiar with and can understand. This implementation of linear regression exists mostly in matrix notation, which keeps it compact and gives us essential flexibility when we start to modify the model. We will start by creating a small dataset and basing our linear regression model on this.

2.2. Creating our notation

	x_1	x_2	y
p_1	5	10	20
p_2	20	25	50
p_3	30	35	70
p_4	50	0	55

Table 2.1: A simple three-dimensional dataset $\rightarrow y = 5 + x_1 + x_2$.

This is our [dataset](#), which consists of four data points: p_1 , p_2 , p_3 , and p_4 . This data takes the form of two inputs, x_1 and x_2 , which are referred to as ‘features’, and one output, y [8]. Recall, from prerequisite knowledge of linear regression, that equations for two-feature data take the form of:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2, \quad (2.1)$$

where $\hat{y}$ has a ‘hat’ to indicate that it is a prediction, and w_0 , w_1 , and w_2 are the parameters of the model, which we solve for. Note: throughout this report, w_i values **always** refer to the estimated parameters, so we reserve ‘hat’ notation for $\hat{y}$ to distinguish them from the true outputs, y . Since we will be working in matrix notation for the majority of the report, we can start by writing the above equation in matrix form. The first step is identifying what matrices and vectors we will need. Naturally, our LHS will be a vector of predictions, $\hat{\mathbf{y}}$, and our RHS will consist of a matrix of all our inputs, $\mathbf{X}$, multiplied by a vector of our parameters, $\mathbf{w}$. Hence we can write the above equation as: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$. The explicit form $\mathbf{X}$ and $\mathbf{w}$, as well as $\mathbf{y}$ to contain the true outputs, are:

$$\mathbf{X} = \begin{bmatrix} 1 & 5 & 10 \\ 1 & 20 & 25 \\ 1 & 30 & 35 \\ 1 & 50 & 0 \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 20 \\ 50 \\ 70 \\ 55 \end{bmatrix}. \quad (2.2)$$

Notice the first column of $\mathbf{X}$ is all 1s, which serves to capture the w_0 parameter for each data point.

In theory, if we found perfect parameters for $\mathbf{w}$, then $\mathbf{X}\mathbf{w}$ would give us exactly $\mathbf{y}$, however, in practice, this is almost never the case, and thus $\mathbf{X}\mathbf{w} = \hat{\mathbf{y}}$. This implies $\mathbf{y} - \hat{\mathbf{y}}$ would give us the error of our predictions, and we can minimise this error by performing a residual sum of squares (RSS) minimisation. We call this error $\boldsymbol{\varepsilon}$.

$$\boldsymbol{\varepsilon} = \mathbf{y} - \hat{\mathbf{y}} = \mathbf{y} - \mathbf{X}\mathbf{w}. \quad (2.3)$$

We can have positive and negative values for the errors in $\boldsymbol{\varepsilon}$, from where our model overshoots or undershoots the true values. We can perform the RSS minimisation (which, notationally, we will refer to as the S function) on this, which squares our errors to make them all positive, thus giving us a degree-2 polynomial that we can minimise.

$$\begin{aligned} S &= \sum_{i=1}^n (\varepsilon_i^2) = \boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) \\ &= (\mathbf{y}^T - \mathbf{w}^T \mathbf{X}^T) (\mathbf{y} - \mathbf{X}\mathbf{w}) \\ &= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X}\mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w} \\ &= \mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w}. \end{aligned} \quad (2.4)$$

where n = the number of data points = 4.

With respect to $\mathbf{w}$, S is a convex quadratic function, which means a minimum exists. We know S is a convex quadratic function because $\mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w} = \|\mathbf{X}\mathbf{w}\|^2 \geq 0$, hence $\mathbf{X}^T \mathbf{X}$ is a positive-semidefinite matrix. To minimise S and find the optimal $\mathbf{w}$, we take the partial derivative of S with respect to $\mathbf{w}$ and set equal to $\mathbf{0}$. This tells us where the quadratic, with respect to $\mathbf{w}$, has a gradient of zero, which is the minimum.

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} S &= \frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X}\mathbf{w}) = \mathbf{0} \\ &= 2\mathbf{X}^T \mathbf{X}\mathbf{w} - 2\mathbf{X}^T \mathbf{y} = \mathbf{0} \\ &\implies \mathbf{X}^T \mathbf{X}\mathbf{w} = \mathbf{X}^T \mathbf{y} \\ &\implies \mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}. \end{aligned} \quad (2.5)$$

Hence, provided $\mathbf{X}^T \mathbf{X}$ is invertible, we can find the optimal parameters inside $\mathbf{w}$ for our linear regression model. To ensure invertibility, we can perform ‘regularisation’ on $\mathbf{X}^T \mathbf{X}$. Regularisation is a technique which involves adding a small value to the main diagonal of $\mathbf{X}^T \mathbf{X}$, making each row linearly independent, and thus ensuring invertibility.

We call this ‘small value’: λ , and we write the regularised version of $\mathbf{X}^T\mathbf{X}$ as $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$, where $\mathbf{I}$ is the identity matrix with the same dimensions as $\mathbf{X}^T\mathbf{X}$. We call this regularisation: ‘Tikhonov regularisation’ [9]. Therefore, the regularised version of the optimal parameter vector is:

$$\mathbf{w} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1} \mathbf{X}^T\mathbf{y}. \quad (2.6)$$

When performing this calculation, $\mathbf{X}^T\mathbf{X}$ turns out to be non-invertible due to linear dependence among the columns, hence the regularisation is necessary. We then calculate $\mathbf{w}$ using a small lambda value: $\lambda = 0.01$, to make $\mathbf{X}^T\mathbf{X}$ invertible, without fundamentally changing the model. The resulting $\mathbf{w}$ vector is:

$$\mathbf{w} = \begin{bmatrix} \frac{14371033769500}{2922809827901} \\ \frac{2927408845000}{2922809827901} \\ \frac{2927744890000}{2922809827901} \end{bmatrix} \approx \begin{bmatrix} 4.917 \\ 1.002 \\ 1.002 \end{bmatrix} \implies \hat{y} = 4.917 + 1.002x_1 + 1.002x_2. \quad (2.7)$$

We can check this against our known [data points](#) to get:

$$p_1 : \hat{y} = 19.947, \quad p_2 : \hat{y} = 50.007, \quad p_3 : \hat{y} = 70.047, \quad p_4 : \hat{y} = 55.017. \quad (2.8)$$

These are all very close to the true outputs of 20, 50, 70, and 55 respectively. If we wanted to make our model more accurate, we could reduce λ to make the regularisation have less impact, but not so small that we introduce numerical instability.

Now that we have created our model, we can use it to predict the output for new inputs. We will be able to verify the accuracy of our model because the true output for any new inputs is: $y = 5 + x_1 + x_2$. Let’s consider the following two new data points:

$$\begin{array}{ccc|ccc} & x_1 & x_2 & y & & x_1 & x_2 & y \\ \hline p_5 & 1 & 72 & 78 & \text{and} & p_6 & -65 & 75 & 15 \end{array} \quad (2.9)$$

which are points our model has never seen, and was not trained on. Our models prediction for p_5 is: $\hat{y} = 78.063$, and for p_6 is: $\hat{y} = 14.937$, which are both very close to the true outputs, even when p_6 has a negative value for x_1 .

This process of performing linear regression with a regularised matrix $\mathbf{X}^T\mathbf{X}$ is known as ‘ridge regression’ [10], and we will be building on this heavily in the next chapter.

Kernel Ridge Regression

3.1. Non-linear data

Consider the following 2D [dataset](#), and [subsequent plot](#), described by the equation $y = x^2$.

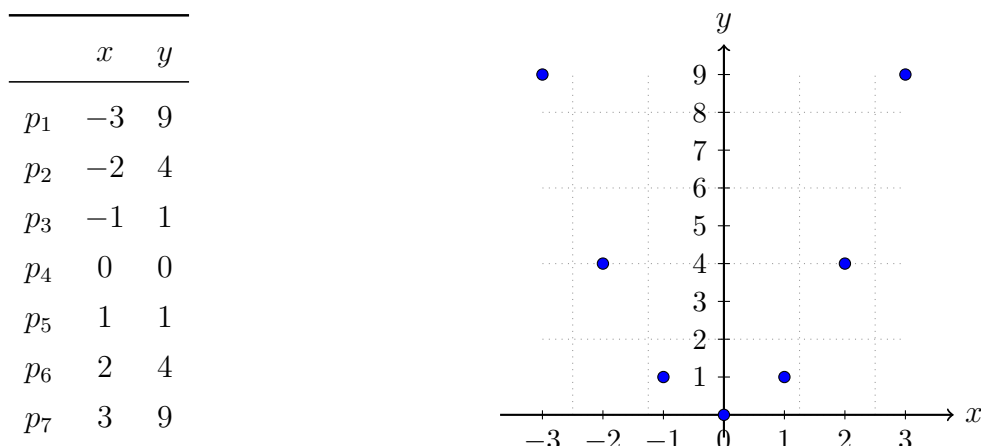


Figure 3.1: A two-dimensional dataset, with its corresponding plot $\rightarrow y = x^2$.

There is clearly a non-linear relationship between x and y , which means a linear regression model would not be adequate for capturing this data. If we attempt to find a linear model for this data, using the same method as in [Ch.2, Sec.2](#), we would get the following equation:

$$\hat{y} = 3.994 + 0x, \quad \text{where } \lambda = 0.01 \quad (3.1)$$

which poorly captures the data. Therefore, if we want to capture this data, we need to use a non-linear model. One way to achieve this is to explicitly create new features in the dataset, based on the original features.

If we look at the 2D [dataset](#), we see that there is only one feature, x , and thus our linear model is of the form: $\hat{y} = w_0 + w_1x$. We can now create a new feature, x^2 , based entirely on the interaction between the original features (e.g. the columns of the $\mathbf{X}$ matrix, in this case: 1 and x). This gives us a new dataset, and a new equation for our model. We will call this increase in features: increasing the ‘degree’. When only finding the first new set of features, we are going from degree 1 to degree 2:

	x	x^2	y	
p_1	-3	9	9	
p_2	-2	4	4	
p_3	-1	1	1	$\rightarrow$
p_4	0	0	0	
p_5	1	1	1	
p_6	2	4	4	
p_7	3	9	9	

$$\begin{aligned} \hat{y} &= w_0(1 \cdot 1) + w_1(1 \cdot x) + w_1(x \cdot 1) + w_2(x \cdot x) \\ &= w_0 + w_1x + w_1x + w_2x^2 \\ &= w_0 + 2w_1x + w_2x^2 \end{aligned} \tag{3.2}$$

Although this non-linear model captures the data perfectly, it presents a problem: what if we had a dataset with 100 data points and 1,000 features (including the bias, w_0 , term)? To do this we must first recognise that the number of features in degree d is given by $(features)^d$.

For this hypothetical dataset, after only creating the second-order of features we would have $1,000^2 = 1,000,000$ total features. We would then have to calculate the 999,000 new features using the original 1,000 features, for all 100 data points, meaning the creation of the $\mathbf{X}$ matrix would take 99,900,000 unique calculations, which is extremely computationally expensive. On top of this, our $\mathbf{X}^T\mathbf{X}$ matrix would be $1,000,000 \times 1,000,000$ in size, meaning it would contain 1,000,000,000,000 (1 trillion) individual values. Needless to say, this becomes infeasible very quickly.

This presents us with two problems to solve:

1. How can we keep $\mathbf{X}^T\mathbf{X}$ a manageable size, without losing the ability to create, and compute, new features?
2. How can we implicitly find $\hat{y}$ for each data point, without explicitly calculating the new features?

3.2. Keeping $\mathbf{X}$ manageable

To achieve [this goal](#), we must start with our equation for $\mathbf{w}$, which we derived in [Ch.2, Sec.2](#): $\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$. We can see that the matrix $\mathbf{X}^T \mathbf{X}$ has variable dimensions: $[\text{features} \times \text{features}]$, which is the source of our problem. A more manageable, fixed, size would be: $[\text{data points} \times \text{data points}]$, which is the size of $\mathbf{X} \mathbf{X}^T$.

We will start by proving that the coefficient of $\mathbf{y}$ in the above equation: $(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T$, can be rewritten as: $\mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1}$

Proof. Let $\mathbf{A}$ be an invertible $n \times n$ matrix, therefore $\mathbf{A}^{-1}$ exists, and $\mathbf{A} \mathbf{A}^{-1} = \mathbf{I}$, where $\mathbf{I}$ is an $n \times n$ identity matrix.

$$\begin{aligned}
 & \mathbf{X}^T = \mathbf{X}^T \quad \text{where } \mathbf{X}^T \text{ is an } m \times n \text{ matrix} \\
 & \underbrace{(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1}}_{m \times m} \underbrace{(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})}_{m \times m} \mathbf{X}^T = \mathbf{X}^T \underbrace{(\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1}}_{n \times n} \underbrace{(\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})}_{n \times n} \\
 & \quad \mathbf{I}_{m \times m} \quad \mathbf{I}_{n \times n} \\
 & (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \underbrace{(\mathbf{X}^T \mathbf{X} \mathbf{X}^T + \lambda \mathbf{I} \mathbf{X}^T)}_{\text{via matrix distributivity}} = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}) \\
 & (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} (\mathbf{X}^T \mathbf{X} \mathbf{X}^T + \lambda \mathbf{X}^T) = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}) \\
 & (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \underbrace{\mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})}_{\text{via matrix distributivity}} = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}) \\
 & (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1}.
 \end{aligned} \tag{3.3}$$

□

Hence, we have shown that our equation for $\mathbf{w}$ can be rewritten as:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} \mathbf{y}. \tag{3.4}$$

This solves our [first problem](#), as we have now formed a matrix with a fixed size. We can now set $\boldsymbol{\alpha} = (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} \mathbf{y}$ which gives us $\mathbf{w} = \mathbf{X}^T \boldsymbol{\alpha}$, and thus:

$$\boldsymbol{\alpha} = (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} \mathbf{y} \implies \hat{\mathbf{y}} = (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I}) \boldsymbol{\alpha} = \mathbf{X} \mathbf{X}^T \boldsymbol{\alpha} \tag{3.5}$$

where $\hat{\mathbf{y}}$ has a hat again to indicate it is a prediction, and $\boldsymbol{\alpha}$ is a vector of parameters we solve for. We also drop $\lambda \mathbf{I}$ in our $\hat{\mathbf{y}}$ equation, because we are no longer inverting, and we now don't need to find $\mathbf{w}$ because it is not present in our new formulation.

Next, we need to solve the [second problem](#): implicitly finding $\hat{y}$ for each data point, without ever explicitly calculating the new features in higher degrees. Take, for example, a dataset with points that each have two features: x_1 and x_2 . Therefore, the columns of $\mathbf{X}$ would be: 1, x_1 , and x_2 . As stated in [Ch.3, Sec.1](#), new features are created from the interaction between the columns of $\mathbf{X}$, meaning our new features (going from degree 1 to degree 2) would be:

$$\begin{aligned} & 1 \cdot 1, \quad 1 \cdot x_1, \quad 1 \cdot x_2, \quad x_1 \cdot 1, \quad x_1 \cdot x_1, \quad x_1 \cdot x_2, \quad x_2 \cdot 1, \quad x_2 \cdot x_1, \quad x_2 \cdot x_2 \\ \implies & \hat{y} = w_0 + 2w_1x_1 + 2w_2x_2 + w_3x_1^2 + 2w_4x_1x_2 + w_5x_2^2 \\ \implies & \hat{y} = w_0 + w_1x_1 + w_1x_1 + w_2x_2 + w_2x_2 + w_3x_1^2 + w_4x_1x_2 + w_4x_1x_2 + w_5x_2^2. \end{aligned} \quad (3.6)$$

The ‘2’ coefficients are present [once again](#), to represent the number of times *that* feature was created. Since we no longer need to find $\mathbf{w}$, as stated [previously](#), we can separate $\hat{y}$ into two parts: $\hat{y} = \mathbf{w}^T \boldsymbol{\phi}_d(p)$, where $\boldsymbol{\phi}_d$ is an arbitrary function that takes a data point, p , and outputs a vector of p ’s features in degree d :

$$\boldsymbol{\phi}_d(p)^T = \begin{bmatrix} 1 & x_1 & x_1 & x_2 & x_2 & x_1^2 & x_1x_2 & x_1x_2 & x_2^2 \end{bmatrix}. \quad (3.7)$$

This allows us to represent $\mathbf{X}$ as a matrix of n separate $\boldsymbol{\phi}_d(p)$ vectors, where $d = 1$ for the original features, $d = 2$ for the degree-2 features, and so on.

$$\begin{aligned} \mathbf{X} &= \begin{bmatrix} - & \boldsymbol{\phi}_d(p_1)^T & - \\ - & \boldsymbol{\phi}_d(p_2)^T & - \\ & \vdots & \\ - & \boldsymbol{\phi}_d(p_n)^T & - \end{bmatrix} \quad \text{which means we can find } \mathbf{X}\mathbf{X}^T \dots \\ & \quad (3.8) \\ \mathbf{X}\mathbf{X}^T &= \begin{bmatrix} \boldsymbol{\phi}_d(p_1)^T \boldsymbol{\phi}_d(p_1) & \boldsymbol{\phi}_d(p_1)^T \boldsymbol{\phi}_d(p_2) & \cdots & \boldsymbol{\phi}_d(p_1)^T \boldsymbol{\phi}_d(p_n) \\ \boldsymbol{\phi}_d(p_2)^T \boldsymbol{\phi}_d(p_1) & \boldsymbol{\phi}_d(p_2)^T \boldsymbol{\phi}_d(p_2) & \cdots & \boldsymbol{\phi}_d(p_2)^T \boldsymbol{\phi}_d(p_n) \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{\phi}_d(p_n)^T \boldsymbol{\phi}_d(p_1) & \boldsymbol{\phi}_d(p_n)^T \boldsymbol{\phi}_d(p_2) & \cdots & \boldsymbol{\phi}_d(p_n)^T \boldsymbol{\phi}_d(p_n) \end{bmatrix}. \end{aligned}$$

This brings us very close to solving both of our [problems](#). The only thing that remains is finding a way to calculate $\boldsymbol{\phi}_d(p_i)^T \boldsymbol{\phi}_d(p_j)$ without explicitly calculating $\boldsymbol{\phi}_d(p)$ for any point p .

To achieve this, we will perform a proof which shows that it is possible to rewrite $\phi_d(p_i)^T \phi_d(p_j)$ in the form $(p_i, p_j)^d$. This requires the following setup:

Setup:

Let $p_i \in \mathbb{R}^n$ where $n = 1 + p$ (i.e. $1 + \aleph$ base features = $\aleph$ columns in $\mathbf{X}$). Now let the components of p_i be indexed by $p_i[k]$ where $k \in \{1, \dots, n\}$. We define $\phi_d(p_i)$ as the vector of all possible products of (d) components of p_i . For $d = 2$, we have:

$$\phi_2(p_i)^T = \left[p_i[k_1] \cdot p_i[k_2] \right], \quad (3.9)$$

which is an ordered vector where $k_1, k_2 \in \{1, \dots, n\}$ form all possible combinations. Hence, for $d = d$, we have:

$$\phi_d(p_i)^T = \left[p_i[k_1] \cdot p_i[k_2] \cdot \dots \cdot p_i[k_d] \right], \quad (3.10)$$

which is also an ordered vector, where $k_1, k_2, \dots, k_d \in \{1, \dots, n\}$ form all possible combinations. Initially $k_1 = k_2 = \dots = k_d = 1$, and we iterate through, starting with k_d , until we have $k_1 = k_2 = \dots = k_d = n$.

Proof.

$$\begin{aligned} \phi_d(p_i)^T \phi_d(p_j) &= \sum_{k_1=1}^n \sum_{k_2=1}^n \dots \sum_{k_d=1}^n (p_i[k_1] \cdot p_i[k_2] \cdot \dots \cdot p_i[k_d]) \\ &= \sum_{k_1=1}^n \sum_{k_2=1}^n \dots \sum_{k_d=1}^n ((p_i[k_1] \cdot p_j[k_1]) \cdot (p_i[k_2] \cdot p_j[k_2]) \cdot \dots \cdot (p_i[k_d] \cdot p_j[k_d])). \end{aligned}$$

$$\begin{aligned} \text{Using the identity: } \sum_{k_1} \sum_{k_2} \dots \sum_{k_d} (f(k_1 \cdot f(k_2 \cdot \dots \cdot f(k_d)))) &= \left(\sum_k (f(k)) \right)^d \\ &= \left(\sum_{k=1}^n (p_i[k] \cdot p_j[k]) \right)^d = (p_i^T p_j)^d. \end{aligned} \quad (3.11)$$

□

Hence, we have shown that $\phi_d(p_i)^T \phi_d(p_j) = (p_i^T p_j)^d$, the well-known polynomial kernel identity [8], and thus:

$$\mathbf{X}\mathbf{X}^T = \begin{bmatrix} (p_1^T p_1)^d & (p_1^T p_2)^d & \dots & (p_1^T p_n)^d \\ (p_2^T p_1)^d & (p_2^T p_2)^d & \dots & (p_2^T p_n)^d \\ \vdots & \vdots & \ddots & \vdots \\ (p_n^T p_1)^d & (p_n^T p_2)^d & \dots & (p_n^T p_n)^d \end{bmatrix}. \quad (3.12)$$

We have now finally solved both of our [problems](#), as we have found a way to keep $\mathbf{X}$ a manageable size, and $(p_i^T p_j)$ allows us to implicitly compute the scalar values of $\phi_d(p_i)^T \phi_d(p_j)$ without explicitly calculating the new features in the higher degrees.

Currently, our p_i vectors contain both the bias term, 1, and the original features. Mathematically, $(p_i^T p_j)^d$, where p_i and p_j contain the bias term, is equivalent to $(1 + p_i^T p_j)^d$, where the bias term has been removed from p_i and p_j . We make this change now because it is a simple adjustment, and will cause less problems with the kernel later on. For clarity, our $\mathbf{X}\mathbf{X}^T$ matrix now looks like this:

$$\mathbf{X}\mathbf{X}^T = \begin{bmatrix} (1 + p_1^T p_1)^d & (1 + p_1^T p_2)^d & \cdots & (1 + p_1^T p_n)^d \\ (1 + p_2^T p_1)^d & (1 + p_2^T p_2)^d & \cdots & (1 + p_2^T p_n)^d \\ \vdots & \vdots & \ddots & \vdots \\ (1 + p_n^T p_1)^d & (1 + p_n^T p_2)^d & \cdots & (1 + p_n^T p_n)^d \end{bmatrix} \quad (3.13)$$

$$\text{where } p_i = \begin{bmatrix} x_{i,1} \\ x_{i,2} \\ \vdots \\ x_{i,p} \end{bmatrix} \quad \text{and} \quad p_j = \begin{bmatrix} x_{j,1} \\ x_{j,2} \\ \vdots \\ x_{j,p} \end{bmatrix}.$$

Now that we have settled on our equation for finding α , and thus $\hat{\mathbf{y}}$, we can compare the number of calculations required to find $\hat{\mathbf{y}}$ for the [hypothetical dataset](#) we discussed on page [10](#).

This dataset has 100 data points and 1,000 features. After using our model to go to the second degree ($d = 2$), we have the equivalent of 1,000,000 features, but we have only performed 10,000 inner products ($\mathbf{X}\mathbf{X}^T$ is 100×100). At this stage, both $\mathbf{X}\mathbf{X}^T$ and the old matrix, $\mathbf{X}^T \mathbf{X}$, would need to be inverted before finding α and $\mathbf{w}$ respectively. Inverting $\mathbf{X}\mathbf{X}^T$, which is 100^2 in size, would require $100^3 = 1,000,000$ computations, because inverting any $n \times n$ matrix, using *LU* decomposition, requires n^3 computations [\[11\]](#). Inverting $\mathbf{X}^T \mathbf{X}$, which is $1,000,000^2$ in size, would require $1,000,000^3 = 1,000,000,000,000,000,000$ (1 quintillion) computations. Hence, at the most computationally expensive stage of each model, our new model is roughly 1,000,000,000,000 (1 trillion) times more efficient.

3.3. The kernel

Now that we have solved both of our [problems](#) from the end of [Ch.3, Sec.1](#), we have finally found the ‘kernel’ [8]. The kernel is a matrix, $\mathbf{K}$, which is equal to $\mathbf{X}\mathbf{X}^T$, but it can also be represented as a function, $K(p_i, p_j)$, which takes two data points, p_i and p_j and finds: $K(p_i, p_j) = (1 + p_i^T p_j)^d$.

Looking at our equation for $\hat{\mathbf{y}}$, we can rewrite this as:

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{X}^T \boldsymbol{\alpha} = \mathbf{K}\boldsymbol{\alpha} \quad \text{where} \quad \boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \hat{\mathbf{y}}. \quad (3.14)$$

Evidently, the predicted value, $\hat{y}$, for any point, p , is found by solving:

$$\begin{aligned} \hat{y} &= (1 + p^T p_1)^d \alpha_1 + (1 + p^T p_2)^d \alpha_2 + \cdots + (1 + p^T p_n)^d \alpha_n \\ &= K(p, p_1) \alpha_1 + K(p, p_2) \alpha_2 + \cdots + K(p, p_n) \alpha_n \\ &= \sum_{i=1}^n (K(p, p_i) \alpha_i), \end{aligned} \quad (3.15)$$

where $\mathbf{K}$ is an $(n \times n)$ matrix, and $\boldsymbol{\alpha}$ is an $(n \times 1)$ vector. This is important, because it means we only calculate the full kernel matrix, $\mathbf{K}$, once, when we are solving for $\boldsymbol{\alpha}$. We then use the kernel function, $K(p, p_i)$ to find the predicted value, $\hat{y}$, for any point, p , which only requires the sum of n computations. To show this, we will utilise the kernel to find predicted values on new data points for the [non-linear dataset](#) we introduced at the start of this [chapter](#).

First, we must choose values for λ and d . We will set $\lambda = 0.01$, which is sufficiently small to have no significant effect on the model, while maintaining computational accuracy (computers can have floating point errors when dealing with extremely small values). We will set $d = 2$, which is the degree we used to create the true y values in the [dataset](#), so our model should perform well.

Since we have defined our kernel mathematically, we can apply these formulas directly in to a programming language of our choice. All calculations in this report are performed in Python, using the NumPy library to store, manipulate, and perform calculations on matrices and vectors. No other libraries are used for calculations.

We start by finding $\mathbf{K}$, which will be a 7×7 matrix, since we have 7 data points. We store our data in NumPy arrays, and find n from the number of data points in our array.

Listing 3.1: Storing data and finding n

```

1 data = [(-3, 9), (-2, 4), (-1, 1), (0, 0), (1, 1), (2, 4),
          (3, 9)]
2 data = np.array(data)
3
4 n = len(data)
5 d = 2
6 lamb = 0.01
7
8 K = np.zeros((n, n))
9 for i in range(n):
10     for j in range(n):
11         K[i, j] = (1 + np.dot(data[i][: -1], data[j][: -1]))
                ** d

```

This means, in just 11 lines of code, we have found our kernel matrix $\mathbf{K}$, which is:

$$\mathbf{K} = \begin{bmatrix} 100 & 49 & 16 & 1 & 4 & 25 & 64 \\ 49 & 25 & 9 & 1 & 1 & 9 & 25 \\ 16 & 9 & 4 & 1 & 0 & 1 & 4 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 4 & 1 & 0 & 1 & 4 & 9 & 16 \\ 25 & 9 & 1 & 1 & 9 & 25 & 49 \\ 64 & 25 & 4 & 1 & 16 & 49 & 100 \end{bmatrix} \implies \boldsymbol{\alpha} = (\mathbf{K} + \lambda \mathbf{I})^{-1} \hat{\mathbf{y}} \approx \begin{bmatrix} 0.0594715 \\ 0.0000678 \\ -0.0355745 \\ -0.4745521 \\ -0.0355745 \\ 0.0000678 \\ 0.0594715 \end{bmatrix}. \quad (3.16)$$

Now that we have our $\boldsymbol{\alpha}$ vector values, we can find the value of $\hat{y}$ for any point. Considering our formula for $\hat{y}$, which is: $\hat{y} = \sum_{i=1}^n (K(p, p_i) \alpha_i)$, we will first find $\hat{y}$ for each of our original data points initial features: $x \in \{-3, -2, -1, 0, 1, 2, 3\}$. This will allow us to see how well our model captures the relationship between x and y in the original data.

For our original data points, we have the following results:

	x	y	$\hat{y}$	
p_1	-3	9	≈ 8.9994052846	where $\hat{y}$ values are rounded to 10 decimal places. (3.17)
p_2	-2	4	≈ 3.9999993221	
p_3	-1	1	≈ 1.0003557446	
p_4	0	0	≈ 0.0004745521	
p_5	1	1	≈ 1.0003557446	
p_6	2	4	≈ 3.9999993221	
p_7	3	9	≈ 8.9994052846	

As we expect, our model captures the original data extremely well. This could be put down to the fact that our model was trained on this data, so let's test how the model performs on unseen data points. We will test the model with four new integer points, two on either side of the original data, as well as the midpoint of each of the original data points. The new x values, their actual y values, and their predicted values, $\hat{y}$, are shown in the table below:

	x	y	$\hat{y}$	% error	
p_8	-6	36	≈ 35.9961974821	0.011%	where $\hat{y}$ values are rounded to 10 decimal places, and % error has been rounded to 3 decimal places. (3.18)
p_9	-4	16	≈ 15.9985736321	0.009%	
p_{10}	-2.5	6.25	≈ 6.2497320052	0.004%	
p_{11}	-1.5	2.25	≈ 2.2502072352	0.009%	
p_{12}	-0.5	0.25	≈ 0.2504448502	0.178%	
p_{13}	0.5	0.25	≈ 0.2504448502	0.178%	
p_{14}	1.5	2.25	≈ 2.2502072352	0.009%	
p_{15}	2.5	6.25	≈ 6.2497320052	0.004%	
p_{16}	5	25	≈ 24.9975043646	0.010%	
p_{17}	7	49	≈ 48.9946529847	0.011%	

This shows that our model performs very well on unseen data points, thus confirming that we have used the kernel to create a regression model that can capture non-linear relationships in data. Hence, we call this process 'kernel ridge regression'. The full code for the calculations in this section can be found in the [Appendices](#) chapter, at the GitHub repository link provided in [Code 1](#).

3.4. Reflection, and what's next?

This section serves as a quick reflection on what we have achieved in this chapter, and how it will shape the rest of the paper.

If we look at the original problem we set out to solve, and the restrictive nature of the previous linear regression model we were using, we can see that we have created an exponentially more powerful model. Our new kernel-based model can capture linear and non-linear relationships in data, meaning it acts as a universal regressor, which is a powerful and flexible tool. Its only limitation is that it does not provide an explicit equation for a line or plane which can be observed and plotted, however, this is a small price to pay for the power, flexibility, and accuracy we have gained.

So far, we have only used the kernel with a λ value of 0.01, and a degree of 2. The kernel can be capable of working with significantly smaller λ values, which would make our kernel ridge regression (KRR) model much more accurate to the training data. It is also capable of utilising much larger d values, which would allow our KRR model to capture 3rd degree, 4th degree, up to infinite degree relationships in data. We will be using these features extensively as we move forward.

The machinery of what we have built up to this point is more powerful than how it might first appear. Currently we have only used the kernel to 'connect' our original data points, by using the kernel to identify the shared properties that link the data points together. However, the kernel can also be used to separate data points. This is the way we will be using the kernel as we progress through to support vector machines (SVMs) which, in our case, will be a type of SVM called a 'least-squares' SVM (LS-SVM).

Here, it must be expressed that the rigorous, difficult, and lengthy mathematics we have gone through, while may have initially seemed like obstacles to show the complexity of our work, will actually be tools that we will repeatedly call upon as we progress.

Our next chapter will be focused on the history of SVMs, and how they came to be. We will then move on to the LS-SVM, how and why they were developed, and what they achieve.

Support Vector Machine History

4.1. Kernels across two centuries

So far, we have covered the mathematics that underpins both linear regression and the formation of the kernel in kernel ridge regression. However, to understand how we will use the kernel in support vector machines, it is useful (and interesting) to take a step back and look at the history of kernels themselves.

The origins of the kernel can be traced as far back as the early 19th century, with the work of Carl Friedrich Gauss. In 1809, Gauss published his book: ‘Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium’ [12]. This book is perhaps best known for formally introducing the Gaussian distribution, but it also contains the error-measuring function: $e^{-h^2\Delta^2}$, where Δ is the observation error and h is a precision parameter. Gauss introduced this function specifically to measure the error in astronomical observations, and it would take mathematicians over 150 years to realise what this function was truly capable of.

If we fast forward to the end of the 19th century, we find that mathematicians who were studying integral equations found that they were repeatedly encountering a particular type of two-variable function. These functions, $\mathbf{K}(x, x')$, were found to be symmetric, and appeared naturally inside many integral operators they were working with. By the start of the 20th century, these functions had become formally established and became known as ‘kernels’ (derived from the German word ‘Kern’, coined by David Hilbert in a series of papers around 1904) [13, 14].

We then move into the early 20th century, where James Mercer, a British mathematician, was also working with integral equations. Particularly, Mercer was interested in the Fredholm integral equations [13], which have the form:

$$\int \mathbf{K}(x, x') f(x') dx' = \lambda f(x). \quad (4.1)$$

His goal was to find a way to show that $\mathbf{K}(x, x')$ could be expressed as the infinite sum of products of simpler functions. The mathematics for *how* this was done are not relevant here, but in 1909 Mercer achieved this, and crucially he proved that $\mathbf{K}(x, x')$ must have a key property: it must be continuous and positive semi-definite [13].

We then make another jump forward to the 1960s when, in 1964, Mark Aizerman, Emmanuel Braverman, and Lev Rozonoer published their paper: ‘Theoretical Foundations of the Potential Function Method in Pattern Recognition Learning’ [15]. In this paper, they tied together the work of Gauss’ error-measuring function, and Mercer’s positive semi-definite kernels condition, to create the ‘potential function method’ [12, 13, 15]. They generalise the Gaussian error function into a more general similarity function for two points: x and x' . This new similarity function [15] takes the form:

$$e^{-\|x-x'\|^2 / 2\sigma^2}. \quad (4.2)$$

Aizerman, Braverman, and Rozonoer showed this equation satisfies Mercer’s condition, that is, it is continuous and positive semi-definite (a formal proof cannot be linked to one particular origin, and did not exist until many years later). They had therefore, inadvertently, created the function that would allow an infinite-dimensional kernel (the Gaussian kernel / RBF kernel), which we will discuss later in the report.

Since these early developments, kernels have continued to be a fundamental part of machine learning. We have already seen how they are derived and used for kernel ridge regression in [Ch.3, Sec.1-3](#), and we will continue to see how they become increasingly important as we move into support vector machines.

4.2. The birth of support vector machines

Support vector machines, at their core, are classification models. That is, they are designed to take in data that has been labelled with classes (usually $+1$ and -1), find a ‘decision boundary’ (a line in 2D, a plane in 3D, a hyperplane in higher dimensions) that separates the classes, and then use this boundary to predict the class of new data points. When data is linearly separable with a decision boundary, mathematicians are able to find infinitely many such boundaries, since any small rotation or shift of a valid boundary still separates classes [16]. This led Vladimir Vapnik to ask the deceptively simple question: *which boundary is the best?* [16]

Classification models have a history that predates support vector machines, stemming back to the 1930s, where Ronald Fisher introduced the ‘Linear Discriminant’ [17]. Fisher was interested in finding a linear boundary that could separate two classes of data, where each class followed a Gaussian distribution (which, as noted, originated from Gauss’ paper we mentioned in the [previous section](#)). Fisher’s method involved finding centroids for each class, and then finding the within-class variance of each class, based on these centroids [17]. Naively, one may attempt to find a decision boundary by simply finding the orthogonal vector of the line connecting the two centroids, but this does not take into account the variance of each class. Fisher’s method, however, does take into account both within-class variance and between-class variance, and finds a unique decision boundary that maximises the ratio of the variances [17].

What Fisher created is a principled and elegant algorithm, with one major flaw: the uniqueness of the decision boundary requires the data must still be described by a Gaussian distribution [17]. Real-world data is rarely perfectly Gaussian, and thus it is not always possible to implement Fisher’s algorithm to achieve a unique decision boundary. However, this is the first time, in any context, that a **unique decision boundary** was found which **considers the variance of the data**.

Jumping forward to the 1950's, in 1958, Frank Rosenblatt published his paper: 'The perceptron: A probabilistic model for information storage and organization in the brain' [18]. In this paper, Rosenblatt formally introduced the perceptron, which is a new type of linear classification model. Rosenblatt's perceptron created a decision boundary by iteratively adjusting the parameters of the linear model, based on errors it made when trying to classify training data. *If* the data was linearly separable, the perceptron would **always** find a decision boundary that cleanly separated the classes [18]. This is an improvement on Fisher's method, which would not necessarily find a separating decision boundary (when one could exist) if the data was non-Gaussian. Fisher's method could instead find a decision boundary with some misclassification.

Since Rosenblatt's perceptron is an iterative algorithm, if no boundary can be found that cleanly separates the classes, the algorithm will never converge, and computations will continue indefinitely. But this is not the only, or biggest, problem with the perceptron. If we have a dataset that *is* linearly separable, then at least one decision boundary will exist, which means there are actually an infinite number of decision boundaries that could separate the classes. The perceptron does not choose the decision boundary that maximises the distance between the boundary and the closest data points (the 'margin'), and instead just chooses the first decision boundary it finds [16]. This is the problem Vladimir Vapnik set out to solve [16].

Through 1963 and 1964, Vapnik, along with his colleagues Alexander Lerner and Alexey Chervonenkis, published multiple papers on the 'Generalized Portrait Algorithm' [19]. This was an early name for what would become the **support vector machine** (SVM). Vapnik co-authored the first paper with Lerner, and the following three with Chervonenkis, and is often considered the main creator of the support vector machine [20]. SVMs take a new approach to finding a decision boundary, by finding the boundary that maximises the margin. In order to do this, the algorithm must first identify the data points from each class that are closest to the other class, and these points are called 'support vectors'. Once these support vectors are chosen, the decision boundary is calculated to be the one that is equidistant from the support vectors, thus maximising the margin [16, 20].

Vapnik and his colleagues realised that if you could find a decision boundary that maximised the margin, you would be able to better classify new data points. This is because a larger margin implies that the decision boundary is further from every data point on both sides. This leaves a much larger gap between where one class ends and another class begins, thus meaning a new data point would have to cross a much greater distance before being misclassified.

At this point, with Vapnik publishing primarily in Russian and working within the Soviet Union, his work on SVMs remained largely inaccessible outside of Russia. Vapnik took the next 15 years developing SVMs further, and in 1979 Vapnik published the original Russian version of: ‘Estimation of Dependences Based on Empirical Data’, later translated into English in 1982, and re-released in 2006 [21]. Up until this point SVMs were only represented in a ‘primal’ formulation, meaning the equation for the hyperplane was expressed directly in terms of the parameters of the decision boundary, e.g. the ‘weight’ vector ‘ $\mathbf{w}$ ’. This will be familiar to us, as it is the [same situation](#) we encountered with kernel ridge regression in [Ch.3, Sec.2](#), where the equation $\mathbf{w} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$ was too computationally expensive to solve when our data had a large number of features. We then showed that this equation could be rewritten as $\mathbf{w} = \mathbf{X}^T (\mathbf{X} \mathbf{X}^T + \lambda \mathbf{I})^{-1} \mathbf{y}$, which was much more computationally efficient. This new form is called the ‘dual’ formulation, and Vapnik’s 1979 paper was the first to show that his SVM algorithm also had a dual formulation [21, 22].

Vapnik formally proved that the dual formulation of the SVM was equivalent to the primal formulation, using Lagrange multipliers and the Karush-Kuhn-Tucker (KKT) conditions [21]. This was a crucial step towards making SVMs a computationally viable algorithm, and, although Vapnik was not yet aware of it, would be the door to allowing the kernel (the same [object we encountered in kernel ridge regression](#)) to be used with SVMs.

4.3. Support vector machines meet the kernel

By the end of 1990, Vapnik was in a position to emigrate to the United States, which allowed him to share his work on support vector machines with the machine learning community in the West [23]. He joined AT&T Bell Labs in Holmdel, New Jersey, and met the very talented team of researchers there, which included: Yoshua Bengio, Bernhard Boser, Léon Bottou, Corinna Cortes, Isabelle Guyon, and Yann LeCun [24].

In 1992, Vapnik, Guyon, and Boser published their paper: ‘A Training Algorithm for Optimal Margin Classifiers’ [25]. Guyon herself wrote, for the reputable data science blog: ‘KDnuggets’, that the story starts with Boser, who, three months from moving away from Bell Labs, implemented Vapnik’s dual formulation of the SVM from 1979. Guyon, noticing that the dual formulation of the SVM only involved dot products of data points, suggested to Boser that he implement the kernel in place of the dot products appearing in the SVMs dual formulation [24].

Vapnik resisted this suggestion at first [24], since the dual formulation of the kernel was created in 1964 by Aizerman, Braverman, and Rozonoer [15], who we mentioned in [Ch.4, Sec.1](#). These were rival Soviet mathematicians who were doing their work around the same time as Vapnik was creating the SVM. Boser implemented the kernel into the SVM anyway, replacing the dot products with a linear kernel, $K(x, x') = x^T x'$ which, being numerically identical to the bare dot product, produced equivalent results. However, the key breakthrough was the ability to then replace the linear kernel, with a polynomial kernel [25].

In 1975, Tomaso Poggio published the paper: ‘On optimal nonlinear associative recall’ [26]. In this paper, Poggio formally introduced the polynomial kernel for the first time, and showed that it satisfies Mercer’s condition [27]. Since the kernel is inherently a pattern-matching function, Poggio was using the polynomial kernel to model how the brain stores and recalls patterns. By the 1990s, the polynomial kernel had become a well-established kernel function, and Guyon recognised this, as well as the framework established by Aizerman, Braverman, and Rozonoer, leading her to recommend that Boser introduce the kernel into his implementation of the SVM [24].

The introduction of the polynomial kernel allowed the SVM to find non-linear decision boundaries between the classes in the original feature space. This was a huge breakthrough, and marks the point where SVMs became a powerful, flexible, and computationally achievable algorithm for classification.

Vapnik, Guyon, and Boser also managed to introduce, in 1992, the Gaussian kernel (also known as the radial basis function (RBF) kernel) into the SVM [25]. Recall that the idea for this kernel was introduced in 1964 by Aizerman, Braverman, and Rozonoer [15], and the Bell Labs team were able to implement and re-write the Gaussian kernel as follows.

$$K(x, x') = \exp(-\gamma \|x - x'\|^2). \quad (4.3)$$

This kernel is particularly powerful in the context of SVMs, because it allows an implicit mapping of the data into an infinite-dimensional feature space, using γ as a radius of influence (which will be discussed in more detail later in the report).

In 1994, researchers Yann LeCun, Christopher Burges, and Corinna Cortes, constructed the ‘Modified National Institute of Standards and Technology database’ or ‘MNIST database’ [28], which was formally published in 1998 [4]. This dataset consists of 70,000 images of handwritten digits from 0 to 9, where 60,000 of the 28×28 pixel images are used for training, and the remaining 10,000 are used for testing. The following year, in 1995, Vapnik and Cortes published a paper that introduced the ‘soft-margin’ SVM, which allowed the SVM to misclassify some data points in the training set (extreme outliers, for example) [29]. In this paper, they used their new soft-margin SVM to achieve a 1.1% error rate on the MNIST dataset [29].

Attaining a 1.1% error rate on the dataset was a significant achievement for support vector machines at the time. It was a clear landmark that showed that SVMs could be used to achieve high performance, and were not just a theoretical algorithm. It remained slightly computationally expensive, and was outperformed by the best neural networks at the time [4], but mathematicians could see its potential. Researchers continued to work on improving SVMs, leading to the development of the ‘least squares’ support vector machine (LS-SVM) [30].

4.4. The least squares support vector machine

By the late 1990s, many classification algorithms had been developed (e.g. random forests, k-nearest neighbours, etc.), and SVMs were increasingly seen as genuinely competitive, thanks to results like the 1.1% error rate on the MNIST dataset [29]. However, SVMs were still computationally expensive, since they required solving a ‘quadratic programming’ problem. This was an iterative process that became more and more burdensome as the size of the dataset increased. In 1999, Johan Suykens and Joos Vandewalle published their paper: ‘Least Squares Support Vector Machine Classifiers’ in the journal ‘Neural Processing Letters’ [30]. In this paper they introduced, what they called, the least squares support vector machine (LS-SVM).

Both the original SVM and the LS-SVM are designed to find a hyperplane that has the form: $f(x) = \mathbf{w}^T \phi(x) + w_0$, where x is a data point, $\phi(x)$ is the arbitrary mapping of the data point to a higher degree, $\mathbf{w}$ (the weight vector) are the coefficients of the hyperplane’s parameters, and w_0 is the bias term. Where the SVM and LS-SVM differ is *how* they find the optimal hyperplane. Formally, the SVM aims to minimise:

$$\min \left(\frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^n \xi_i \right). \quad (4.4)$$

subject to the constraints: $y_i \cdot f(x_i) \geq 1 - \xi_i$ and $\xi_i \geq 0$, where ξ_i are slack variables we choose that allow margin violations (i.e. how soft the margin is). C is a regularisation parameter we also choose that controls the trade-off between maximising the margin and minimising the classification error (larger C means a harder margin; smaller C means a softer margin).

The α values are then found by solving the quadratic programming problem:

$$\max \left(\sum_{i=1}^n (\alpha_i) - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i \alpha_j y_i y_j K(x_i, x_j)) \right). \quad (4.5)$$

subject to the constraints: $0 \leq \alpha_i \leq C$ and $\sum_{i=1}^n \alpha_i y_i = 0$. Solving for α is an iterative process that is very computationally expensive.

The LS-SVM, on the other hand, aims to minimise:

$$\begin{aligned}\min(S) &= \min(\boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} + \gamma \mathbf{w}^T \mathbf{w}) \\ &= \min(\boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} + \gamma \boldsymbol{\alpha}^T \mathbf{K} \boldsymbol{\alpha}).\end{aligned}\tag{4.6}$$

This is subject to the constraints: $y_i \cdot f(x_i) = \mathbf{w}^T \boldsymbol{\phi}(x_i) + w_0 = 1 - \varepsilon_i$, where ε_i are the unconstrained error variables, and γ is the regularisation parameter we choose (large γ means we penalise large α_i values more heavily, and vice versa). Since the only constraint is an equality constraint, we are able to minimise S by solving a least squares minimisation problem, and we can find the optimal $\boldsymbol{\alpha}$ by solving a system of linear equations. This substitution of $\boldsymbol{\alpha}^T \mathbf{K} \boldsymbol{\alpha}$ is non-trivial and a preview of what we will derive and compute in the following chapter.

Because we are able to solve for $\boldsymbol{\alpha}$ by solving a system of linear equations, the LS-SVM becomes much more computationally efficient than the original SVM. A LS-SVM is also inherently a soft-margin SVM, since the error variables ε_i are unconstrained, and thus can be any real value in order to minimise S .

The LS-SVM does have one major drawback, which is that it makes every data point a support vector. The original SVM algorithm allowed for some α_i values to be zero, which meant that the corresponding data point, x_i , did not contribute to the decision boundary, and thus was not a support vector. This is in contrast to the LS-SVM, which doesn't try to make the α_i values zero for well-classified data points. Hence, the LS-SVM can be more prone to 'overfitting', which means the model is trying too hard to perfectly fit the training data, leading to a well-conformed model that can be bad at classifying new data [31].

The LS-SVM does have a way to mitigate this issue, by introducing a 'weighting' term for the error variables [31]. If we have a data point with a large error (positive or negative), it is likely to be an outlier, and thus we can give it a smaller weight to reduce its influence on the decision boundary. On the other hand, if we have a data point with a small error, it is very close to the margin, and thus we can give it a larger weight. The process of choosing these weights is straightforward, and involves an iterative process. This, naturally, makes the LS-SVM slightly more computationally expensive, but what it loses in speed, it gains in mathematical elegance and interpretability.

The weighted LS-SVM was introduced by Suykens and Vandewalle, along with co-authors Jos de Brabanter and Lukas Lukas, in their 2002 paper: ‘Weighted least squares support vector machines: robustness and sparse approximation’ [31]. This paper introduced the weighted LS-SVM and showed that, since the kernel would no longer guaranteed to be positive definite, it could be solved using the ‘conjugate gradient method’, which is an iterative method for solving systems of linear equations. This reduces the computational cost of the weighted LS-SVM from $\mathcal{O}(n^3)$ to $\mathcal{O}(nr^2)$ for each iteration [31], where n is the number of datapoints, and r is the effective rank of the kernel matrix. Introducing this particular linear solver is outside the scope of this report, since it reduces the mathematical interpretability of the model, but it is worth noting for completeness.

Mathematicians have continued to work on LS-SVMs, starting with the 2002 book: ‘Least Squares Support Vector Machines’ by Suykens, Vandewalle, and de Brabanter, as well as co-authors Bart De Moor and Tony Van Gestel [32]. This book provides a comprehensive overview of the LS-SVM, solidifying least squares support vector machines as a mature and established method for data classification.

In the following chapter, we will pick up where we left off with the kernel in our kernel ridge regression equations (Ch.3), and derive the equations for the LS-SVM. We will then incorporate the weighted LS-SVM, and apply it to a real-world dataset.

Least Squares SVMs

5.1. Derivation of the LS-SVM

To begin the derivation of the least squares support vector machine, we must first define some training data that requires an SVM in order to separate.

	x_1	x_2	y
p_1	0	0	+1
p_2	0.5	0	+1
p_3	0.35	0.35	+1
p_4	0	0.5	+1
p_5	-0.35	0.35	+1
p_6	-0.5	0	+1
p_7	-0.35	-0.35	+1
p_8	0	-0.5	+1
p_9	0.35	-0.35	+1

	x_1	x_2	y
p_{10}	1	0	-1
p_{11}	0.87	0.5	-1
p_{12}	0.5	0.87	-1
p_{13}	0	1	-1
p_{14}	-0.5	0.87	-1
p_{15}	-0.87	0.5	-1
p_{16}	-1	0	-1
p_{17}	-0.87	-0.5	-1
p_{18}	-0.5	-0.87	-1
p_{19}	0	-1	-1
p_{20}	0.5	-0.87	-1
p_{21}	0.87	-0.5	-1

Table 5.1: LS-SVM bullseye training data.

A plot of this data can be seen on the following page, where the blue points are the positive class (the left table) and the red points are the negative class (the right table).

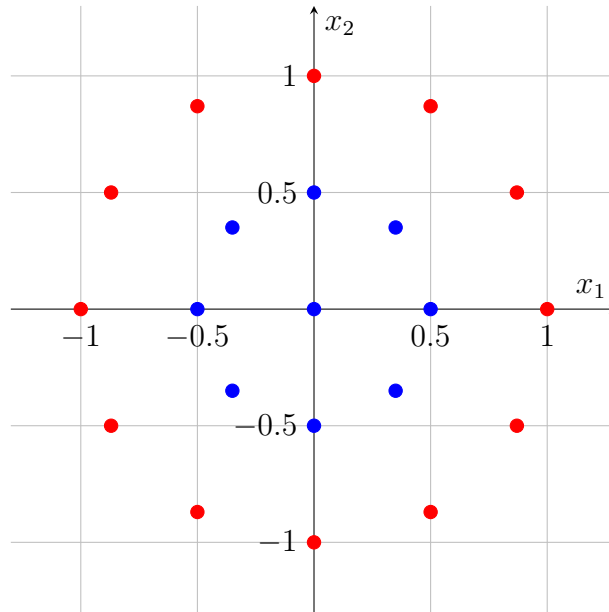


Figure 5.1: A bullseye dataset. Blue: +1 class; Red: -1 class.

Using the data from [Table 5.1](#), we have produced [Figure 5.1](#) which represents a ‘bullseye’ dataset [\[33\]](#). The y values of the data points are either +1 or -1, which indicates the two classes. Our goal is to find a hyperplane that can separate the two classes, and then use this hyperplane to predict the class of new data points. It is clear that the hyperplane that will best separate the two classes (i.e. the one with the largest margin) will be a circle centred at the origin, with a radius of ≈ 0.75 .

In kernel ridge regression, we aimed to find a hyperplane that would pass through the data points, or at least get very close to them. Recall that our KRR hyperplane was derived from the linear regression equation [\(2.1\)](#) from [Ch.2, Sec.2](#).

$$f(p_i) = w_0 + w_1 x_{i,1} + w_2 x_{i,2}. \quad (5.1)$$

We then established that this hyperplane could be rewritten in matrix notation:

$$\mathbf{f}(p) = \mathbf{X}\mathbf{w}, \quad (5.2)$$

where $\mathbf{X}$ was a matrix where the rows were the data points, and $\mathbf{w}$ was a vector of parameters. Both including the bias term.

Later, in the [KRR chapter](#), we showed that this could be broken down further, into the form:

$$f(p_i) = \mathbf{w}^T \boldsymbol{\phi}(p_i) + w_0. \quad (5.3)$$

$\boldsymbol{\phi}(p_i)$ is the arbitrary increase in degree of the data point p_i , and w_0 is the outcome of removing the bias term from the data point vector, $\boldsymbol{\phi}(p_i)$. In our current example, with no increase in degree, our vectors are:

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \quad \text{and} \quad \boldsymbol{\phi}(p_i) = \begin{bmatrix} x_{i,1} \\ x_{i,2} \end{bmatrix}. \quad (5.4)$$

At this stage, we are ready to start forming the conditions for the hyperplane. Since we still want our hyperplane to act as a prediction function, we want it to satisfy:

$$\begin{aligned} f(p_i) &= +1 & \text{when} & \quad \text{class}(p_i) = y_i = +1, \\ f(p_i) &= -1 & \text{when} & \quad \text{class}(p_i) = y_i = -1. \end{aligned} \quad (5.5)$$

Hence, it follows that:

$$y_i \cdot f(p_i) = y_i \cdot (\mathbf{w}^T \boldsymbol{\phi}(p_i) + w_0) = 1. \quad (5.6)$$

The result of this is that a perfect hyperplane would have the property that every data point, p_i , would have an absolute distance of 1 from the hyperplane.

Unfortunately, a perfect hyperplane is rarely possible, and so we introduce error variables, ε_i . These variables are individual for each data point, and give us a concrete value to minimise later on.

$$y_i \cdot f(p_i) = y_i \cdot (\mathbf{w}^T \boldsymbol{\phi}(p_i) + w_0) = 1 - \varepsilon_i. \quad (5.7)$$

To understand why the sign of ε_i is negative, we can rearrange the equation.

$$\varepsilon_i = 1 - y_i \cdot f(p_i). \quad (5.8)$$

If f correctly classifies p_i , then $y_i \cdot f(p_i)$ will be positive, and so $|\varepsilon_i|$ will be ≤ 1 for all well-classified points where $|f(p_i)| \leq 2$. However, if f misclassifies p_i , then $y_i \cdot f(p_i)$ will be negative, leading to $|\varepsilon_i| \geq 1$ for all $f(p_i)$. Therefore, by making the RHS of equation (5.7): $1 - \varepsilon_i$, $|\varepsilon_i|$ (and thus ε_i^2) values are naturally inflated for misclassified points, which is a fact we will take advantage of when we minimise ε_i .

Now that we have introduced the error variables, and established that it has the correct sign, we can rearrange equation (5.8).

$$\begin{aligned} y_i \cdot \varepsilon_i &= y_i - f(p_i) \\ &= y_i - \mathbf{w}^T \boldsymbol{\phi}(p_i) - w_0. \end{aligned} \tag{5.9}$$

This is now in a form which we can rewrite in matrix notation. y_i becomes a vector of y values: $\mathbf{y}$, and ε_i becomes a vector of error values for each data point: $\boldsymbol{\varepsilon}$. This implies that the LHS of the equation becomes: $\mathbf{y} \odot \boldsymbol{\varepsilon}$, where $\odot$ is the element-wise product. For the RHS, we define the rows of $\mathbf{X}$ to be the $\boldsymbol{\phi}(p_i)$ vectors, which reflects the definition of $\mathbf{X}$ in equation (3.8) from Ch.3, Sec.2. Naturally, our new equation becomes:

$$\mathbf{y} \odot \boldsymbol{\varepsilon} = \mathbf{y} - \mathbf{X}\mathbf{w} - w_0\mathbf{1}, \tag{5.10}$$

where $\mathbf{1}$ is an n -dimensional vector of 1s, which serves to capture the bias term.

We can rearrange this equation further, specifically by rewriting $\mathbf{w}$ in a form that will allow us to introduce the kernel. Recall from equation (3.4) in Ch.3, Sec.2, that $\mathbf{w} = \mathbf{X}^T \boldsymbol{\alpha}$. This particular incarnation of $\mathbf{X}^T$ had a row of 1s to capture the bias term, so this can be removed due to our above equation already having a separate bias term. Hence, we can rewrite $\mathbf{y} \odot \boldsymbol{\varepsilon}$ as:

$$\mathbf{y} \odot \boldsymbol{\varepsilon} = \mathbf{y} - \mathbf{X}\mathbf{X}^T \boldsymbol{\alpha} - w_0\mathbf{1}, \tag{5.11}$$

and since the kernel matrix is specifically defined as $\mathbf{K} = \mathbf{X}\mathbf{X}^T$ in equation (3.14), we can rewrite this as:

$$\mathbf{y} \odot \boldsymbol{\varepsilon} = \mathbf{y} - \mathbf{K}\boldsymbol{\alpha} - w_0\mathbf{1}. \tag{5.12}$$

We now have a form of the equation which can be minimised by performing a least-squares minimisation, which is, functionally, very similar to the RSS minimisation we performed in linear regression.

We start by noticing that the LHS vector, $\mathbf{y} \odot \boldsymbol{\varepsilon}$, is a vector of the variables:

$$\begin{bmatrix} y_1 \cdot \varepsilon_1 & y_2 \cdot \varepsilon_2 & \cdots & y_n \cdot \varepsilon_n \end{bmatrix}^T. \quad (5.13)$$

Finding the sum of the squares of these variables gives us:

$$\begin{aligned} S &= (y_1 \varepsilon_1)^2 + (y_2 \varepsilon_2)^2 + \cdots + (y_n \varepsilon_n)^2 \\ &= \varepsilon_1^2 + \varepsilon_2^2 + \cdots + \varepsilon_n^2 \\ &= \boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} \end{aligned} \quad (5.14)$$

because $y_i^2 = 1$ for all i . Therefore, our least squares minimisation is equivalent to minimising the sum of squares of the error variables, and $\mathbf{y}$ disappears from the LHS. We can continue our derivation of S by performing a sum of squares on the RHS.

$$\begin{aligned} S &= \boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} \\ &= (\mathbf{y} - \mathbf{K}\boldsymbol{\alpha} - w_0 \mathbf{1})^T (\mathbf{y} - \mathbf{K}\boldsymbol{\alpha} - w_0 \mathbf{1}) \\ &= \mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{K}\boldsymbol{\alpha} - 2w_0 \mathbf{y}^T \mathbf{1} + 2w_0 \boldsymbol{\alpha}^T \mathbf{K}^T \mathbf{1} + \boldsymbol{\alpha}^T \mathbf{K}^T \mathbf{K} \boldsymbol{\alpha} + w_0^2 \mathbf{1}^T \mathbf{1} \\ &= \mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{K}\boldsymbol{\alpha} - 2w_0 \mathbf{y}^T \mathbf{1} + 2w_0 \boldsymbol{\alpha}^T \mathbf{K} \mathbf{1} + \boldsymbol{\alpha}^T \mathbf{K}^2 \boldsymbol{\alpha} + w_0^2 \mathbf{1}^T \mathbf{1}. \end{aligned} \quad (5.15)$$

$\mathbf{K}$ is a symmetric matrix, so $\mathbf{K}^T = \mathbf{K}$, and $\mathbf{K}^T \mathbf{K} = \mathbf{K}^2$. Here, S appears as a convex quadratic function of both unknowns $\boldsymbol{\alpha}$ and w_0 , which means a minimum exists for both. We find these minima by taking partial derivatives of S with respect to $\boldsymbol{\alpha}$ and w_0 , and setting them equal to 0.

$$\begin{aligned} \frac{\partial S}{\partial \boldsymbol{\alpha}} &= -2(\mathbf{y}^T \mathbf{K})^T + 2\mathbf{K}^2 \boldsymbol{\alpha} + 2w_0 \mathbf{K} \mathbf{1} = 0 \\ \implies \mathbf{K}\boldsymbol{\alpha} + w_0 \mathbf{1} &= \mathbf{y}, \quad \text{when } \mathbf{K} \text{ is invertible} \end{aligned} \quad (5.16)$$

$$\frac{\partial S}{\partial w_0} = -2\mathbf{y}^T \mathbf{1} + 2\boldsymbol{\alpha}^T \mathbf{K} \mathbf{1} + 2w_0 \mathbf{1}^T \mathbf{1} = 0. \quad (5.17)$$

We can ensure that $\mathbf{K}$ is invertible by adding the regularisation term, $\lambda \mathbf{I}$, and we require it to be invertible for future use. This means that we can substitute y from (5.16) into (5.17), which gives us:

$$\begin{aligned} \frac{\partial S}{\partial w_0} &= -2\boldsymbol{\alpha}^T \mathbf{K} \mathbf{1} - 2w_0 \mathbf{1}^T \mathbf{1} + 2\boldsymbol{\alpha}^T \mathbf{K} \mathbf{1} + 2w_0 \mathbf{1}^T \mathbf{1} = 0 \\ \implies 0 &= 0. \end{aligned} \quad (5.18)$$

Unfortunately, this result indicates that $\frac{\partial S}{\partial \alpha}$ and $\frac{\partial S}{\partial w_0}$ are linearly dependent, meaning that there are infinitely many pairs (α, w_0) that minimise S . This means there is no unique solution that minimises S . This is a problem, since we want to find a unique solution that minimises S , which will then give us a unique, best hyperplane.

The solution is to introduce a new term to S that will break the linear dependence of the partial derivatives. We can consider $\mathbf{w} = \mathbf{X}^T \alpha$ again, and observe that $\mathbf{w}$ is a vector that contains coefficients that are applied to the features of the data points. If a value in $\mathbf{w}$ is large, then the corresponding feature has a large influence on the predicted value, and vice versa. If all values in $\mathbf{w}$ are large, then the model becomes very complex, and sensitive to small changes in the data. In this case, our model would be overfitting. Similarly, if all values in $\mathbf{w}$ are small, then the model becomes too simple, and would therefore be underfitting. By finding the scalar value: $\mathbf{w}^T \mathbf{w}$, we can capture the overall complexity of the model, and by adding this to S , we can penalise complex models, and thus encourage simpler models.

To start, we rewrite $\mathbf{w}^T \mathbf{w}$ in more convenient terms.

$$\mathbf{w}^T \mathbf{w} = (\mathbf{X}^T \alpha)^T \mathbf{X}^T \alpha = \alpha^T \mathbf{X} \mathbf{X}^T \alpha = \alpha^T \mathbf{K} \alpha. \quad (5.19)$$

Now, we can add this to S :

$$S = \epsilon^T \epsilon + \alpha^T \mathbf{K} \alpha, \quad (5.20)$$

and finally we add a coefficient, γ , to the new term, so that we can control its influence on the minimisation of S .

$$\begin{aligned} S &= \epsilon^T \epsilon + \gamma \alpha^T \mathbf{K} \alpha \\ &= (\mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1})^T (\mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1}) + \gamma \alpha^T \mathbf{K} \alpha \\ &= \mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T \mathbf{K} \alpha - 2w_0 \mathbf{y}^T \mathbf{1} + 2w_0 \alpha^T \mathbf{K} \mathbf{1} + \alpha^T \mathbf{K}^2 \alpha + w_0^2 \mathbf{1}^T \mathbf{1} + \gamma \alpha^T \mathbf{K} \alpha. \end{aligned} \quad (5.21)$$

We now take the partial derivatives of S with respect to α and w_0 again. The partial derivative with respect to α will have an extra term, and the partial derivative with respect to w_0 will be unchanged, which will break the linear dependence.

$$\begin{aligned}
\frac{\partial S}{\partial \boldsymbol{\alpha}} &= -2(\mathbf{y}^T \mathbf{K})^T + 2\mathbf{K}^2 \boldsymbol{\alpha} + 2w_0 \mathbf{K} \mathbf{1} + 2\gamma \mathbf{K} \boldsymbol{\alpha} = 0 \\
&= -2\mathbf{K} \mathbf{y} + 2\mathbf{K}^2 \boldsymbol{\alpha} + 2w_0 \mathbf{K} \mathbf{1} + 2\gamma \mathbf{K} \boldsymbol{\alpha} = 0 \\
&= \mathbf{K} \boldsymbol{\alpha} + w_0 \mathbf{1} + \gamma \boldsymbol{\alpha} = \mathbf{y}, \quad \text{when } \mathbf{K} \text{ is invertible} \\
\downarrow \quad \text{rearranging} \quad &\implies \boxed{\gamma \boldsymbol{\alpha} = \mathbf{y} - \mathbf{K} \boldsymbol{\alpha} - w_0 \mathbf{1} = \mathbf{y} \odot \boldsymbol{\varepsilon}} \\
&= (\mathbf{K} \boldsymbol{\alpha} + \gamma \boldsymbol{\alpha}) + w_0 \mathbf{1} = \mathbf{y} \\
&= (\mathbf{K} + \gamma \mathbf{I}) \boldsymbol{\alpha} + w_0 \mathbf{1} = \mathbf{y}, \\
&\text{and since } \mathbf{K} \text{ is of the form } \mathbf{K} + \lambda \mathbf{I}, \text{ we combine } \gamma \text{ and } \lambda \\
&= (\mathbf{K} + \lambda \mathbf{I}) \boldsymbol{\alpha} + w_0 \mathbf{1} = \mathbf{y}.
\end{aligned} \tag{5.22}$$

$$\begin{aligned}
\frac{\partial S}{\partial w_0} &= -2\mathbf{y}^T \mathbf{1} + 2\boldsymbol{\alpha}^T \mathbf{K} \mathbf{1} + 2w_0 \mathbf{1}^T \mathbf{1} = 0 \\
&= \mathbf{y}^T \mathbf{1} - \boldsymbol{\alpha}^T \mathbf{K} \mathbf{1} - w_0 \mathbf{1}^T \mathbf{1} = 0 \\
&= (\mathbf{y}^T - \boldsymbol{\alpha}^T \mathbf{K} - w_0 \mathbf{1}^T) \mathbf{1} = 0 \\
&= (\mathbf{y} - \mathbf{K} \boldsymbol{\alpha} - w_0 \mathbf{1})^T \mathbf{1} = 0 \\
&= (\mathbf{y} \odot \boldsymbol{\varepsilon})^T \mathbf{1} = 0 \implies \mathbf{1}^T (\mathbf{y} \odot \boldsymbol{\varepsilon}) = 0 \\
&= \mathbf{1}^T \gamma \boldsymbol{\alpha} = 0 \implies \mathbf{1}^T \boldsymbol{\alpha} = 0.
\end{aligned} \tag{5.23}$$

Hence, we can now represent $\frac{\partial S}{\partial \boldsymbol{\alpha}}$ and $\frac{\partial S}{\partial w_0}$ as two linearly independent equations, meaning that there is now one unique pair $(\boldsymbol{\alpha}, w_0)$ that minimises S . We can take equation (5.22) and (5.23) and write them as a system of equations in matrix form.

$$\begin{bmatrix} \mathbf{K} + \lambda \mathbf{I} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha} \\ w_0 \end{bmatrix} = \begin{bmatrix} \mathbf{y} \\ 0 \end{bmatrix}. \tag{5.24}$$

We know the LHS matrix (which is a ‘bordered’ matrix, due to the border of $\mathbf{1}$ vectors), and the RHS vector, so we can solve for $\boldsymbol{\alpha}$ and w_0 using standard linear algebra solving techniques. Fundamentally, we have derived everything needed to use the least squares support vector machine. In the next section, we will actually use this LS-SVM on the bullseye dataset, and see how it performs on the training data, and on new data points.

5.2. Applying the LS-SVM to real data

To apply the LS-SVM on real data, we must start with equation (5.12), which we rewrite in index notation:

$$\begin{aligned}
 \mathbf{y} \odot \boldsymbol{\varepsilon} &= \mathbf{y} - \mathbf{K}\boldsymbol{\alpha} - w_0 \mathbf{1} \implies y_i \cdot \varepsilon_i = y_i - (\mathbf{K}\boldsymbol{\alpha})_i - w_0 \\
 &\implies y_i - f(p_i) = y_i - (\mathbf{K}\boldsymbol{\alpha})_i - w_0 \\
 &\implies f(p_i) = (\mathbf{K}\boldsymbol{\alpha})_i + w_0 \\
 &\implies f(p_i) = \sum_{j=1}^n (k(p_i, p_j) \alpha_j) + w_0.
 \end{aligned} \tag{5.25}$$

Here, we use the identity $y_i \cdot \varepsilon_i = y_i - f(p_i)$, which is established in equation (5.9), which we combine with our knowledge of the structure of $\mathbf{K}$ defined in equation (3.13), since $\mathbf{K} = \mathbf{X}\mathbf{X}^T$. This means that once we have obtained $\boldsymbol{\alpha}$ and w_0 , we then use our [new formula](#) for $f(p_i)$ to make predictions for new data points, p_i .

To begin, we create our matrix, $\mathbf{K}$, and regularise it with $\lambda \mathbf{I}$, where we set $\lambda = 0.01$. Since our data is very clean (two circles), our λ value makes very little difference, so we just ensure it is small enough that our model is not underfitting, which is where the model is too simple to capture the structure of the data. Keeping λ around 0.01 is often a good starting point for tuning λ values, since smaller values *can* lead to overfitting and/or numerical instability for more complex datasets. The last variable to select is d , which is the degree of the polynomial kernel. We can choose $d = 2$, which will be sufficient to capture the circular structure of the data, but we could increase d if our data had a more complex structure. A portion of our bordered matrix can be seen below:

$$\begin{bmatrix} \mathbf{K} + \lambda \mathbf{I} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} = \begin{bmatrix} 1.01 & 1 & 1 & \cdots & 1 \\ 1 & 1.5725 & 1.380625 & \cdots & 1 \\ 1 & 1.380625 & 1.560025 & \cdots & 1 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & 1 & \cdots & 0 \end{bmatrix}. \tag{5.26}$$

At this stage we are able to solve for α and w_0 . This requires our bordered matrix to be invertible, which *is the case* here as long as λ is > 0 . This ensures our regularised kernel is positive definite, since $\mathbf{K}$ is positive semidefinite, and adding λ to the diagonal increases each eigenvalue by λ . The next step is to perform LU decomposition on the bordered matrix, which has time complexity of $\mathcal{O}(n^3)$ [11], where n is the number of data points. This keeps the LS-SVM computationally efficient, and reduces the chance of floating point errors that can arise from directly calculating the inverse of matrices [34] (which would also have time complexity of $\mathcal{O}(n^3)$ [11]). The following are some of the values of α and the w_0 value that we obtain from solving this linear algebra problem:

$$\alpha \approx \begin{bmatrix} -52.15617849 \\ 10.10299396 \\ 8.85781051 \\ \vdots \\ -3.11948872 \\ -1.40113556 \\ -1.40113556 \end{bmatrix} \quad \text{and} \quad w_0 \approx 1.52156178. \quad (5.27)$$

The final step is to use these values to predict the class of new data points. A good starting point is to predict the class of the training data. We can begin by showing the calculation for the first data point, p_1 , where we want our prediction $f(p_1)$ to be close to $+1$, and more importantly, to be greater than 0 (since the sign of $f(p_1)$ is the predicted class).

$$\begin{aligned} f(p_1) &= \sum_{j=1}^n (k(p_1, p_j) \alpha_j) + w_0 \\ &\approx k(p_1, p_1)(-52.156) + k(p_1, p_2)(10.103) + \cdots + k(p_1, p_{21})(-1.401) + 1.522 \\ &\approx (1 + p_1^T p_1)(-52.156) + \cdots + (1 + p_1^T p_{21})(-1.401) + 1.522 \\ &\approx -52.156 + 10.103 + \cdots - 1.401 + 1.522 \\ &\approx 1.522 \text{ (3 d.p.)}. \end{aligned} \quad (5.28)$$

This is a good result, since $f(p_1)$ is greater than 0, and close to $+1$. We can apply the ‘*sign*’ function to $f(p_1)$ to get the predicted class: $\text{sign}(f(p_1)) = +1$, which is the correct class. We can repeat this process for all the training data points, and we achieve the results shown in Figure A.1 in the appendix.

The LS-SVM classifies all the training data points correctly, which is exactly what we would expect. The point at the origin, p_1 is significantly further from the hyperplane than the other points, and therefore has a much higher predicted f value than the other positive-class points. The remaining predictions, for both classes, are not constant, which reflects the rounding errors present in the actual dataset (for example, $(0.35, 0.35)$ is not exactly on the circle of radius 0.5).

Now that we can confirm the LS-SVM is correctly classifying the training data, we now must see how it performs on a set of new, unseen, data points. The tables and plot below show the new data points.

	x_1	x_2	y
p_{22}	0.5	0.5	?
p_{23}	0	-0.6	?
p_{24}	0.35	-0.25	?

	x_1	x_2	y
p_{25}	-1	1	?
p_{26}	-0.75	0.5	?
p_{27}	-0.71	-0.71	?

Table 5.2: New bullseye data points.

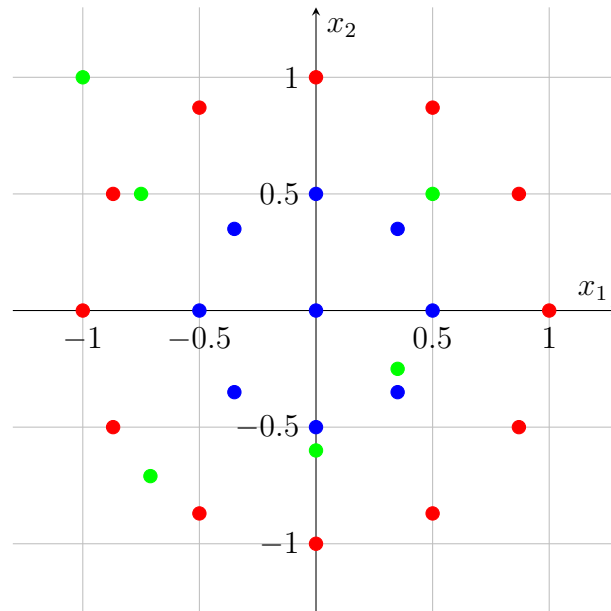


Figure 5.2: The bullseye dataset. Blue: +1 class; Red: -1 class; Green: new points.

By inspection of the plot, we can see that p_{23} , p_{24} , p_{25} , p_{26} , and p_{27} belong to classes $+1$, $+1$, -1 , -1 , and -1 respectively. p_{22} appears to be very close to where we expect the decision boundary to be, so it is not immediately clear which class it belongs to, but analytically, we can find that p_{22} *should* belong to the $+1$ class. The results of the prediction function are shown in the table below.

	x_1	x_2	$f(p_i)$ to 3 d.p.	$\text{sign}(f(p_i))$
p_{22}	0.5	0.5	0.276	$+1$
p_{23}	0	-0.6	0.625	$+1$
p_{24}	0.35	-0.25	1.061	$+1$
p_{25}	-1	1	-3.459	-1
p_{26}	-0.75	0.5	-0.502	-1
p_{27}	-0.71	-0.71	-0.989	-1

Table 5.3: Predictions for new bullseye data points.

Therefore, our LS-SVM correctly classifies all the new data points, and does so with an acceptable distance from the hyperplane for all points (including p_{22} , which is 0.276 units from the hyperplane). The full code for this section's calculations can be found in the appendices chapter under [Code 2](#).

In the following section, we will introduce the concept of the ‘weighted’ LS-SVM, which is a variant of the LS-SVM which we spoke about in [Ch.4, Sec.4](#). The weightings will be applied to the error variables, ε_i , which will allow us to give importance to certain data points, so that the model is less sensitive to outliers.

5.3. The weighted LS-SVM

The weighted LS-SVM (wLS-SVM), which is sometimes referred to as the ‘iteratively reweighted’ LS-SVM [35], is formed by simply adding a weighing term to the error variables in our least squares minimisation. We then solve α and w_0 , find our errors in our predictions, and update the weights based on these errors. The iterative process of the wLS-SVM converges [31], and from testing we have performed, convergence is often achieved in less than 20 iterations. The wLS-SVM often leads to an improvement in performance on new data points compared to the standard LS-SVM [31].

To start, we must first introduce some data that is less clean than the [bullseye dataset](#), which contains some outliers. Consider the following dataset, which is simple, and symmetrical, apart from the three severe outliers.

Table 5.4: Class: +1				Table 5.5: Class: ?				Table 5.6: Class: -1			
	x_1	x_2	y		x_1	x_2	y		x_1	x_2	y
p_1	-2.5	1.5	+1	p_{14}	1	1	?	p_{20}	2.5	1.5	-1
p_2	-3.5	1.5	+1	p_{15}	0.5	1.5	?	p_{21}	3.5	1.5	-1
p_3	-3.5	0.5	+1	p_{16}	-0.5	1.5	?	p_{22}	3.5	0.5	-1
p_4	-2.5	0.5	+1	p_{17}	-1	1	?	p_{23}	2.5	0.5	-1
p_5	-3	1	+1	p_{18}	-0.5	0.5	?	p_{24}	3	1	-1
p_6	-6	2	+1	p_{19}	0.5	0.5	?	p_{25}	6	2	-1
p_7	-5	1	+1					p_{26}	5	1	-1
p_8	-4	0	+1					p_{27}	4	0	-1
p_9	-4	2	+1					p_{28}	4	2	-1
p_{10}	-6	0	+1					p_{29}	6	0	-1
p_{11}	5.5	1.5	+1								
p_{12}	5	0.5	+1								
p_{13}	6	1	+1								

Figure 5.3: Outlier dataset for wLS-SVM.

The resulting plot of the data in Figure 5.3 is shown below.

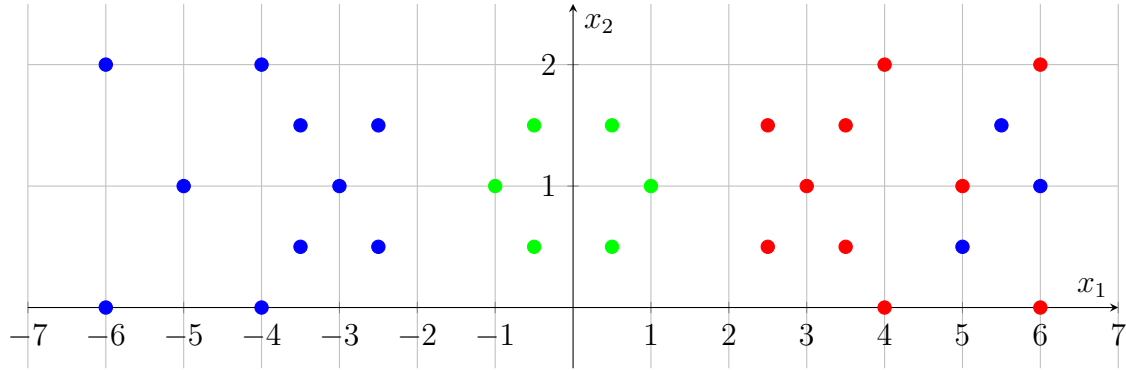


Figure 5.4: Outlier dataset. Blue: +1 class; Green: unknown; Red: -1 class.

Clearly, in this dataset, p_{11} , p_{12} , and p_{13} are the outliers, since they are deep within the -1 class' territory. A wLS-SVM will be able to give these points low weights, so let's derive the wLS-SVM, and see how it performs on this dataset.

We begin by creating a matrix, $\mathbf{V}$, which is a diagonal matrix that contains the weights for each data point.

$$\mathbf{V} = \begin{bmatrix} v_1 & 0 & \cdots & 0 \\ 0 & v_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & v_n \end{bmatrix}. \quad (5.29)$$

Initially, we choose $v_i = 1$ for all i , so $\mathbf{V} = \mathbf{I}$. We can then look at our least squares minimisation equation, S , and attempt to apply the weights to it.

$$\begin{aligned} S &= \boldsymbol{\varepsilon}^T \boldsymbol{\varepsilon} + \gamma \boldsymbol{\alpha}^T \mathbf{K} \boldsymbol{\alpha} \\ &= \boldsymbol{\varepsilon}^T \mathbf{I} \boldsymbol{\varepsilon} + \gamma \boldsymbol{\alpha}^T \mathbf{K} \boldsymbol{\alpha} \\ &= \boldsymbol{\varepsilon}^T \mathbf{V} \boldsymbol{\varepsilon} + \gamma \boldsymbol{\alpha}^T \mathbf{K} \boldsymbol{\alpha}. \end{aligned} \quad (5.30)$$

This is why the initial weights are set to 1, since it means the introduction of $\mathbf{V}$ does not change the value of S at the start. We can see that if a value in $\mathbf{V}$ is small, then the corresponding error variable will have a much smaller influence on the value of S , meaning our model will be less sensitive to the data point that relates to that error.

We now need to take the partial derivatives of S with respect to α and w_0 again, to see how this change has affected the equations. Let's start by rewriting S .

$$\begin{aligned}
S &= \varepsilon^T \mathbf{V} \varepsilon + \gamma \alpha^T \mathbf{K} \alpha \\
&= (\mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1})^T \mathbf{V} (\mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1}) + \gamma \alpha^T \mathbf{K} \alpha \\
&= (\mathbf{y}^T - \alpha^T \mathbf{K} - w_0 \mathbf{1}^T) \mathbf{V} (\mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1}) + \gamma \alpha^T \mathbf{K} \alpha \\
&= (\mathbf{y}^T \mathbf{V} - \alpha^T \mathbf{K} \mathbf{V} - w_0 \mathbf{1}^T \mathbf{V}) (\mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1}) + \gamma \alpha^T \mathbf{K} \alpha \\
&= \mathbf{y}^T \mathbf{V} \mathbf{y} - \mathbf{y}^T \mathbf{V} \mathbf{K} \alpha - w_0 \mathbf{y}^T \mathbf{V} \mathbf{1} - \alpha^T \mathbf{K} \mathbf{V} \mathbf{y} + \alpha^T \mathbf{K} \mathbf{V} \mathbf{K} \alpha + \\
&\quad w_0 \alpha^T \mathbf{K} \mathbf{V} \mathbf{1} - w_0 \mathbf{1}^T \mathbf{V} \mathbf{y} + w_0^2 \mathbf{1}^T \mathbf{V} \mathbf{1} + \gamma \alpha^T \mathbf{K} \alpha \\
&= \mathbf{y}^T \mathbf{V} \mathbf{y} - 2\mathbf{y}^T \mathbf{V} \mathbf{K} \alpha - 2w_0 \mathbf{y}^T \mathbf{V} \mathbf{1} + 2w_0 \alpha^T \mathbf{K} \mathbf{V} \mathbf{1} + \alpha^T \mathbf{K} \mathbf{V} \mathbf{K} \alpha + \\
&\quad w_0^2 \mathbf{1}^T \mathbf{V} \mathbf{1} + \gamma \alpha^T \mathbf{K} \alpha.
\end{aligned} \tag{5.31}$$

Which allows us to compute $\frac{\partial S}{\partial \alpha}$ and $\frac{\partial S}{\partial w_0}$.

$$\begin{aligned}
\frac{\partial S}{\partial \alpha} &= (-2\mathbf{y}^T \mathbf{V} \mathbf{K})^T + 2w_0 \mathbf{K} \mathbf{V} \mathbf{1} + 2\mathbf{K} \mathbf{V} \mathbf{K} \alpha + 2\gamma \mathbf{K} \alpha = 0 \\
&= -2\mathbf{K} \mathbf{V} \mathbf{y} + 2w_0 \mathbf{K} \mathbf{V} \mathbf{1} + 2\mathbf{K} \mathbf{V} \mathbf{K} \alpha + 2\gamma \mathbf{K} \alpha = 0 \\
&= -\mathbf{V} \mathbf{y} + w_0 \mathbf{V} \mathbf{1} + \mathbf{V} \mathbf{K} \alpha + \gamma \alpha = 0, \quad \text{where } \mathbf{K} \text{ is invertible} \\
&= \mathbf{V} \mathbf{K} \alpha + \gamma \alpha + w_0 \mathbf{V} \mathbf{1} = \mathbf{V} \mathbf{y} \\
&= \mathbf{K} \alpha + \gamma \mathbf{V}^{-1} \alpha + w_0 \mathbf{1} = \mathbf{y}, \quad \text{where } \mathbf{V} \text{ is invertible} \\
&\downarrow \quad \text{rearranging} \quad \implies \quad \boxed{\gamma \mathbf{V}^{-1} \alpha = \mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1} = \mathbf{y} \odot \varepsilon} \\
&= (\mathbf{K} + \gamma \mathbf{V}^{-1}) \alpha + w_0 \mathbf{1} = \mathbf{y} \\
&= (\mathbf{K} + \lambda \mathbf{V}^{-1}) \alpha + w_0 \mathbf{1} = \mathbf{y}, \quad \text{where we combine regularisation terms.}
\end{aligned} \tag{5.32}$$

$$\begin{aligned}
\frac{\partial S}{\partial w_0} &= -2\mathbf{y}^T \mathbf{V} \mathbf{1} + 2\alpha^T \mathbf{K} \mathbf{V} \mathbf{1} + 2w_0 \mathbf{1}^T \mathbf{V} \mathbf{1} = 0 \\
&= \mathbf{y}^T \mathbf{V} \mathbf{1} - \alpha^T \mathbf{K} \mathbf{V} \mathbf{1} - w_0 \mathbf{1}^T \mathbf{V} \mathbf{1} = 0 \\
&= (\mathbf{y}^T - \alpha^T \mathbf{K} - w_0 \mathbf{1}^T) \mathbf{V} \mathbf{1} = 0 \\
&= (\mathbf{y} - \mathbf{K} \alpha - w_0 \mathbf{1})^T \mathbf{V} \mathbf{1} = 0 \\
&= (\gamma \mathbf{V}^{-1} \alpha)^T \mathbf{V} \mathbf{1} = 0 \\
&= (\mathbf{1}^T \mathbf{V}) \gamma \mathbf{V}^{-1} \alpha = 0 \quad \implies \quad \gamma \mathbf{1}^T \mathbf{V} \mathbf{V}^{-1} \alpha = 0 \\
&= \gamma \mathbf{1}^T \alpha = 0 \quad \implies \quad \mathbf{1}^T \alpha = 0.
\end{aligned} \tag{5.33}$$

Our results from (5.32) and (5.33) give us two linearly independent equations, allowing us to solve for $\boldsymbol{\alpha}$ and w_0 in a similar way to the standard LS-SVM.

$$\begin{bmatrix} \mathbf{K} + \lambda \mathbf{V}^{-1} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha} \\ w_0 \end{bmatrix} = \begin{bmatrix} \mathbf{y} \\ 0 \end{bmatrix}. \quad (5.34)$$

Where this linear algebra problem has $\boldsymbol{\varepsilon}^T \mathbf{V} \boldsymbol{\varepsilon}$ implicitly baked in.

Initially, $\mathbf{V}$ has only 1s on the diagonal, so: $\mathbf{V} = \mathbf{I} = \mathbf{V}^{-1}$. This means that our first iteration of the wLS-SVM is exactly the same as the standard LS-SVM, where we use LU decomposition on the bordered matrix, and solve for $\boldsymbol{\alpha}$ and w_0 . Looking back at equation (5.25), we can see that the prediction function for the training data points is as follows.

$$\begin{aligned} f(p_i) &= (\mathbf{K}\boldsymbol{\alpha})_i + w_0 \\ \implies \mathbf{f} &= \mathbf{K}\boldsymbol{\alpha} + w_0 \mathbf{1}. \end{aligned} \quad (5.35)$$

Since we have $\mathbf{y}$ and $\mathbf{f}$, we can compute the magnitude of the errors by performing element-wise subtraction.

$$|\boldsymbol{\varepsilon}| = |\mathbf{y} - \mathbf{f}| \implies |\boldsymbol{\varepsilon}| = |\mathbf{y} - \mathbf{K}\boldsymbol{\alpha} - w_0 \mathbf{1}|. \quad (5.36)$$

If the error is large, we want the corresponding weight to be small, so we set the weights to be inversely proportional to the magnitude of the errors:

$$v_i = \frac{1}{|\varepsilon_i| + \delta}, \quad (5.37)$$

where δ is a small constant added to prevent division by zero. Since the environment of $\mathbf{V}$ is: $\boldsymbol{\varepsilon}^T \mathbf{V} \boldsymbol{\varepsilon}$, it is desirable to have the squared errors in the denominator of v_i . This allows us to set $\mathbf{V}$ as follows.

$$\mathbf{V} = \begin{bmatrix} \frac{1}{\varepsilon_1^2 + \delta} & 0 & \cdots & 0 \\ 0 & \frac{1}{\varepsilon_2^2 + \delta} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{\varepsilon_n^2 + \delta} \end{bmatrix}. \quad (5.38)$$

Squaring the errors eliminates the need to find the magnitude, $|\varepsilon_i|$. We then solve equation (5.34) again with the new $\mathbf{V}$, and repeat this process until convergence.

In our linear system, we only ever need to compute $\mathbf{V}^{-1}$, which is trivial since $\mathbf{V}$ is a diagonal matrix. Therefore, we just generate $\mathbf{V}^{-1}$ explicitly, every time, without needing to compute the inverse of $\mathbf{V}$.

$$\mathbf{V}^{-1} = \begin{bmatrix} \varepsilon_1^2 + \delta & 0 & \cdots & 0 \\ 0 & \varepsilon_2^2 + \delta & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \varepsilon_n^2 + \delta \end{bmatrix}. \quad (5.39)$$

The δ still remains important, since if any of the errors are 0, the δ prevents the corresponding weight from becoming infinite.

As stated at the start of [this section](#), our testing showed that the wLS-SVM often converges in less than 20 iterations, so this process of updating $\mathbf{V}$ is not too computationally expensive. To show the model performance we can gain from using the wLS-SVM, compared to the standard LS-SVM, we can apply both models to the [outlier dataset](#), using the same λ value and kernel degree for both. The full predictions for both models are shown in the appendices in [Figure A.2](#), and the resulting plots are shown in [Figure A.3](#) and [Figure A.4](#).

[Figure A.3](#) and [Figure A.4](#) are presented such that, data points that have a known class, keep their original class, regardless of the model's prediction, and data points with an unknown class are assigned a class based on what the model predicts. By observing these plots, we can see that the standard LS-SVM is heavily influenced by the outliers, and therefore misclassifies two of the unknown data points, p_{16} and p_{18} . On the other hand, the wLS-SVM is able to nullify the influence of the outliers by assigning them low weights, and therefore correctly classifies all six of the unknown data points.

By looking at the raw predictions for the LS-SVM in [Table A.3](#), we can see that our LS-SVM is predicting p_{13} to be in the +1 class which, although technically correct, is not ideal since p_{13} is a very extreme outlier. The wLS-SVM ([Table A.4](#)), by contrast, predicts p_{11} , p_{12} , and p_{13} to be in the -1 class, while still managing to correctly classify all the data points it should.

When working with real-world data, outliers are common, and are often much more difficult to identify than in our [outlier dataset](#), so the wLS-SVM is a very useful version of the LS-SVM to be able to use in situations where the standard LS-SVM is not performing well. An alternative to the wLS-SVM is to use a higher degree polynomial kernel, which can allow the model to fit the data better. In some cases, this can lead to better performance, but in other cases, it can lead to overfitting, and worse performance on new data points. For our [outlier dataset](#), there was no rhyme or reason to the outliers, so a higher degree polynomial kernel would have attempted to fit the outliers better, when instead they should just be ignored.

We can see how heavily the wLS-SVM penalises the outliers by looking at the final weights assigned to each data point. The full set weightings for each point are shown in [Table A.5](#), and links to the full code used to compute the regular LS-SVM and wLS-SVM are shown in [Code 3](#) and [Code 4](#), respectively. From the final weights, we can see that our wLS-SVM model has assigned extremely high weights to points p_2 , p_3 , p_5 , p_{19} , p_{21} , p_{22} , and p_{23} . This means our model considers these points to be very important for determining the decision boundary, and will therefore be very sensitive to these points. In contrast, the outliers, p_{11} , p_{12} , and p_{13} , have been assigned the lowest three weights, which are significantly smaller than many of the other points' weights. This means our model recognises these points as outliers, and has reduced their influence on the positioning of the decision boundary.

In the next chapter, we will derive the infinite dimensional kernel (which we will refer to as the 'radial basis function' (RBF) kernel [\[36\]](#)), and apply it to the MNIST dataset [\[4\]](#).

wLS-SVM model for MNIST dataset

6.1. Creating our infinite degree polynomial kernel

To apply the wLS-SVM model we have developed to the MNIST dataset (often referred to as the MNIST database [4]), we must first understand what the MNIST dataset contains, and how the data is structured. The MNIST dataset consists of 70,000 greyscale images of handwritten arabic digits (0 - 9) with a resolution of 28×28 pixels. Of these 70,000 images, 60,000 are used for training our model, and the remaining 10,000 are used for testing it [4]. The training and testing sets are always the same, so that we can compare our results directly with other models that have been trained on the same data [4].

Due to the large size of the dataset, it is common practice to use a subset of the training data [4, 37, 38], allowing us to train our model in a reasonable amount of time. In our case, a stratified random selection of different sizes of the training data are used, ranging from 1,000 total training points (1.67%) to 30,000 total training points (50%). The stratified random selection ensures that we have an equal number of training points from each class (digit) in each subset. We also apply our model to the full 60,000 training set points, which does not have equal numbers of data points for each class. The training set contains between $\sim 5,400$ and $\sim 6,800$ training points per class, which we found by directly analysing the raw MNIST data.

The data is structured such that each image is represented as a vector of 784 integer values from 0 to 255, where each value corresponds to the light intensity of a pixel in the image. We normalise these values to the range $[0, 1]$, 0 represents white/black, 1 represents black/white, and the values in-between represent different shades of grey. We perform this normalisation because data points with many large pixel values will dominate in the kernel function and grow too large, which could lead to numerical instability.

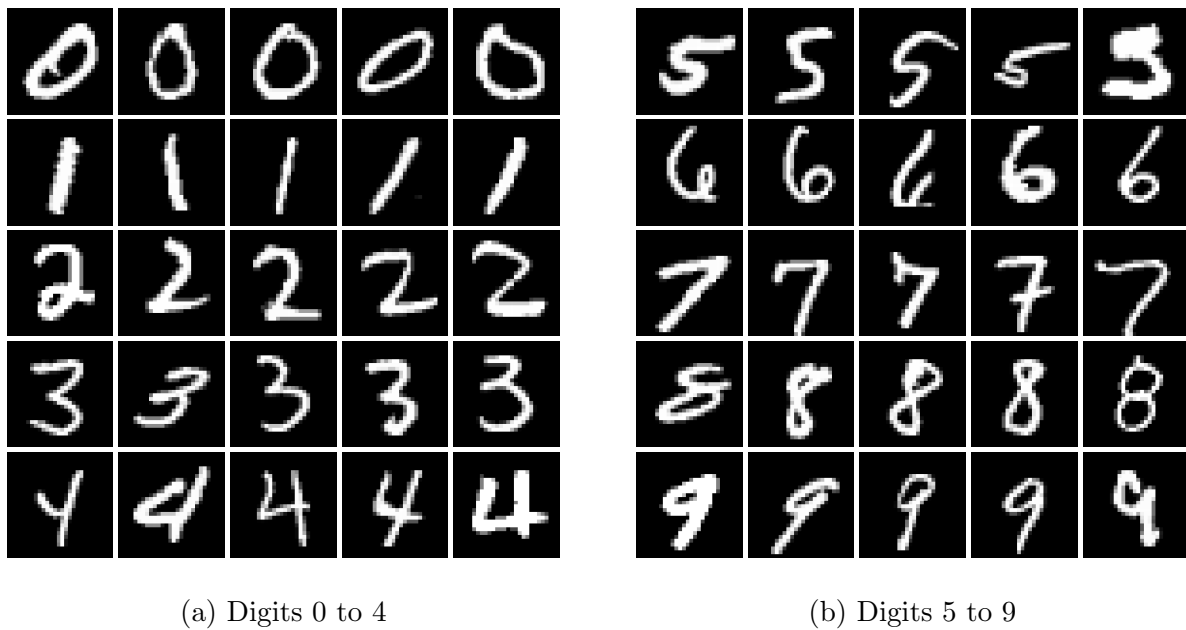


Figure 6.1: A sample of 50 data points from the MNIST training set, with 5 examples for each class (digit). Both plot (a) and plot (b) were generated by the author, from raw MNIST data, using the Python library: Matplotlib. We choose 0 for black and 1 for white for the pixel values to enhance readability, but our data remains unchanged.

Here, in [Figure 6.1](#), we can see that the data points can be vastly different from each other, even within the same class. For example, in [Figure 6.1b](#), ‘6’ is written similarly for columns 2, 4, and 5, but columns 1 and 3 (particularly column 3) look very different. Points can also look very similar across classes, for example the ‘4’ in column 2 of [Figure 6.1a](#) looks very similar to some of the ‘9’s in [Figure 6.1b](#), and some of the ‘2’s in [Figure 6.1a](#) appear similar to the number ‘7’. There are, of course, differences between these similar points, and it is the job of our model to recognise these subtleties and classify unseen points, which could have ambiguous features, correctly.

To give our model the ‘power’ to be able to classify these points correctly, we need the kernel inside our wLS-SVM model to be able to capture all of the possible relationships between the data points. As stated in [Ch.3, Sec.2](#), solving $(1 + p_i^T p_j)^d$ implicitly captures all degree d relationships between the data points, from degree = 0 (the ‘1’ term on its own) to degree = d (the product of d features).

However, this creates a problem: the number of new features that are created by going from degree d to degree $d + 1$ is $(features)^{d+1} - (features)^d$. For example, if we have $features = 784$ (which is how many features we have in the MNIST dataset), and we go from degree $d = 20$ to degree $d = 21$, we create $784^{21} - 784^{20} \approx 6.03 \times 10^{60}$ new implicit features. The new features that are products of 21 features, make up the largest proportion of these new features. These are solved implicitly, which means the calculation is very efficient, but the result will be heavily influenced by the higher degree features. If we want to push d higher, e.g. tending towards $d = \infty$, in order to capture all possible relationships between the data points, linear, quadratic, and cubic relationships will be drowned out by the higher degree relationships, and we have no way of controlling this.

We can look at the equation $(1 + p_i^T p_j)^d$, where $d \rightarrow \infty$, as a sum of infinitely many terms, from degree = 0 to degree = ∞ .

$$\sum_{d=0}^{\infty} (1 + p_i^T p_j)^d. \quad (6.1)$$

This has the same problem as just setting d to a very high value, in that the higher degree terms will dominate the sum. However, now that we have an infinite sum, we can introduce a coefficient, c_d , for each term in the sum.

$$\sum_{d=0}^{\infty} (c_d (1 + p_i^T p_j)^d). \quad (6.2)$$

This c_d term gives us two benefits:

1. We can reduce the influence of the higher degree terms by making c_d smaller as d increases.
2. We can make the sum converge, so that we have a scalar value which represents the similarity between two data points in infinite-degree space.

Now that we have an infinite sum which converges, we need to find the value of c_d for each d . To do this, we turn to the Taylor series expansion of the exponential function.

The Taylor series expansion of e^z , as we know, is given by:

$$e^z = \sum_{d=0}^{\infty} \left(\frac{z^d}{d!} \right), \quad (6.3)$$

which converges for all $z \in \mathbb{R}$. If we let $z = 1 + p_i^T p_j$, we can now write the following.

$$e^{1+p_i^T p_j} = \sum_{d=0}^{\infty} \left(\frac{(1 + p_i^T p_j)^d}{d!} \right) = \sum_{d=0}^{\infty} \left(\frac{1}{d!} (1 + p_i^T p_j)^d \right). \quad (6.4)$$

This gives us a good starting point for our value of c_d : $c_d = \frac{1}{d!}$. We can then introduce a new parameter, γ (since our first γ was absorbed by λ in the wLS-SVM derivation). When implemented, our sum will look like this:

$$\sum_{d=0}^{\infty} \left(\frac{\gamma^d}{d!} (1 + p_i^T p_j)^d \right) = e^{\gamma(1+p_i^T p_j)}. \quad (6.5)$$

This γ parameter allows us to directly control the influence of the higher degree terms in the sum. If γ is small, the denominator ($d!$) dominates γ^d more quickly, meaning the higher d is, the smaller c_d will be, reducing the influence of higher degree terms. Similarly, if γ is large, it takes longer for γ^d to dominate, meaning higher degree terms will have more of an impact on the sum.

Since γ is a scalar parameter that we can tune, we can choose our numerator to be $(2\gamma)^d$ instead of γ^d , in order to help clean up our calculations later on. This gives our sum the following form:

$$\sum_{d=0}^{\infty} \left(\frac{(2\gamma)^d}{d!} (1 + p_i^T p_j)^d \right) = e^{2\gamma(1+p_i^T p_j)}. \quad (6.6)$$

Therefore, we have managed to capture all possible relationships between two data points in an infinite degree polynomial kernel, and express it in a closed form. However, this form still requires some further manipulation.

We can expand the exponential function as follows:

$$e^{2\gamma(1+p_i^T p_j)} = e^{2\gamma} \cdot e^{2\gamma p_i^T p_j}. \quad (6.7)$$

Since this effectively represents row- i , column- j of the kernel matrix, and γ is a fixed parameter we choose, then $e^{2\gamma}$ is a constant that scales the entire kernel matrix. By looking at the wLS-SVM in matrix form (equation 5.34), we can see that scaling the kernel matrix by any constant leads to α absorbing this value.

$$\text{Scaling } \mathbf{K} \rightarrow c\mathbf{K}, \text{ leads to } \alpha \rightarrow \frac{1}{c}\alpha. \quad (6.8)$$

This means $e^{2\gamma}$ has no effect on the final model, and it can be dropped from our kernel function calculations. Therefore, we currently write our kernel function as follows:

$$K(p_i, p_j) = e^{2\gamma p_i^T p_j} = \sum_{d=0}^{\infty} \left(\frac{(2\gamma)^d}{d!} (p_i^T p_j)^d \right). \quad (6.9)$$

We now utilise the vector identity:

$$\begin{aligned} \|p_i - p_j\|^2 &= \|p_i\|^2 - 2p_i^T p_j + \|p_j\|^2 \\ \implies 2p_i^T p_j &= \|p_i\|^2 + \|p_j\|^2 - \|p_i - p_j\|^2 \\ \implies 2\gamma p_i^T p_j &= \gamma\|p_i\|^2 + \gamma\|p_j\|^2 - \gamma\|p_i - p_j\|^2 \\ \implies e^{2\gamma p_i^T p_j} &= e^{\gamma\|p_i\|^2 + \gamma\|p_j\|^2 - \gamma\|p_i - p_j\|^2} \\ \implies e^{2\gamma p_i^T p_j} &= e^{\gamma\|p_i\|^2} \cdot e^{\gamma\|p_j\|^2} \cdot e^{-\gamma\|p_i - p_j\|^2}. \end{aligned} \quad (6.10)$$

Since we know that the LHS of the above equation expands to represent the elements of an infinite degree polynomial kernel, we must consider which parts of the RHS are responsible for finding the same relationships between the data points. It is clear that $e^{\gamma\|p_i\|^2}$ is independent of $e^{\gamma\|p_j\|^2}$, and they are both positive, non-zero constants that scale $e^{-\gamma\|p_i - p_j\|^2}$. If p_i and p_j are both substantially far from the origin, but are not close to each other, then $e^{\gamma\|p_i\|^2}$ and $e^{\gamma\|p_j\|^2}$ will be large. This inflates the value of $e^{-\gamma\|p_i - p_j\|^2}$, which is small since its value is representative of the raw distance between p_i and p_j . Similarly, if p_i and p_j are small, but close to each other, $e^{-\gamma\|p_i - p_j\|^2}$ will be large, but will be deflated by $e^{\gamma\|p_i\|^2}$ and $e^{\gamma\|p_j\|^2}$. This is undesirable, since we do not want our kernel function to be influenced by the distance of the data points from the origin, and instead only by the distance between the data points themselves.

Thanks to these observations, we can look at our exponential function again:

$$e^{2\gamma p_i^T p_j} = e^{\gamma \|p_i\|^2} \cdot e^{\gamma \|p_j\|^2} \cdot e^{-\gamma \|p_i - p_j\|^2}. \quad (6.11)$$

Since the LHS term: $e^{2\gamma p_i^T p_j}$, is a scaled version of the RHS term: $e^{-\gamma \|p_i - p_j\|^2}$ (the only RHS term that measures the similarity between p_i and p_j), we can rewrite our kernel function as shown below.

$$K(p_i, p_j) = e^{-\gamma \|p_i - p_j\|^2} = \exp(-\gamma \|p_i - p_j\|^2). \quad (6.12)$$

Readers will recognise this as the Gaussian kernel, which we discussed in [Ch.4, Sec.3](#), equation (4.3). This form was proposed by Vapnik, Guyon and Boser in 1992 [25], and was originally developed by Aizerman, Braverman, and Rozonoer in 1964 [15]. The 1992 paper identified that a kernel of this form satisfied Mercer's condition, and could be used in SVMs to capture infinite degree relationships between data points. This is the mathematical backbone we need in order to use this kernel (commonly known as the 'radial basis function' (RBF) kernel) in our wLS-SVM model.

Now that we have reached the final form of our RBF kernel, we can rewrite this new kernel in matrix notation. To begin, we create a vector, $\mathbf{q}$, where $q_i = \|p_i\|^2$ for $i = 1, \dots, n$ with n being the number of data points, and we write:

$$\begin{aligned} K(p_i, p_j) &= e^{-\gamma \|p_i - p_j\|^2} = \exp(-\gamma \|p_i - p_j\|^2) \\ &= \exp(-\gamma (\|p_i\|^2 + \|p_j\|^2 - 2p_i^T p_j)) \\ &= \exp^\odot(-\gamma (\mathbf{q}\mathbf{1}^T + \mathbf{1}\mathbf{q}^T - 2\mathbf{X}\mathbf{X}^T)) \\ &= \exp^\odot(\gamma (2\mathbf{X}\mathbf{X}^T - \mathbf{q}\mathbf{1}^T - \mathbf{1}\mathbf{q}^T)). \end{aligned} \quad (6.13)$$

where ' $\exp^\odot$ ' is the element-wise exponential function of the matrix: $\gamma (2\mathbf{X}\mathbf{X}^T - \mathbf{q}\mathbf{1}^T - \mathbf{1}\mathbf{q}^T)$, rows of $\mathbf{X}$ do not contain the bias term, and $\mathbf{1}$ is an n -dimensional vector of ones.

This is the form of the kernel function that will be used in our wLS-SVM model for the MNIST dataset. Our next goal is to find a way to make our wLS-SVM model capable of classifying data points into 1 of 10 classes, rather than just 1 of 2 classes, which is what we have been doing so far.

6.2. Multi-class classification

To achieve multi-class classification with our wLS-SVM model, we can use the ‘one-vs-rest’ (OvR) strategy [39]. This process involves breaking the training data into 10 different sets, where each set contains a different class (digit) for the positive class, and all other classes are combined together for the negative class.

	+1 class	−1 class
Set 1	0	1, 2, 3, 4, 5, 6, 7, 8, 9
Set 2	1	0, 2, 3, 4, 5, 6, 7, 8, 9
Set 3	2	0, 1, 3, 4, 5, 6, 7, 8, 9
Set 4	3	0, 1, 2, 4, 5, 6, 7, 8, 9
Set 5	4	0, 1, 2, 3, 5, 6, 7, 8, 9
Set 6	5	0, 1, 2, 3, 4, 6, 7, 8, 9
Set 7	6	0, 1, 2, 3, 4, 5, 7, 8, 9
Set 8	7	0, 1, 2, 3, 4, 5, 6, 8, 9
Set 9	8	0, 1, 2, 3, 4, 5, 6, 7, 9
Set 10	9	0, 1, 2, 3, 4, 5, 6, 7, 8

Table 6.1: Example of the one-vs-rest strategy for the MNIST dataset.

We simply treat each set as a separate classification problem for our wLS-SVM model, and produce 10 different models. Each model shares the same kernel matrix, but has different values in $\mathbf{y}$, and thus, different $\boldsymbol{\alpha}$ and w_0 values.

In order to make a prediction for an unseen data point, the calculation is the same as equation (5.35).

$$\mathbf{f} = \mathbf{K}\boldsymbol{\alpha} + w_0\mathbf{1}. \quad (6.14)$$

We do this calculation 10 times, once for each model, and we end up with 10 different predictions. Whichever model gives the highest prediction value, is the model that is ‘most confident’ that the unseen data point belongs to its positive class, and therefore we classify the unseen data point as belonging to that class. This is equivalent of taking the ‘argmax’ of the 10 different prediction values, since the argmax function gives us the index of the maximum value in a vector, and typical vector implementations start at index 0 (particularly in programming languages).

Below is the full mathematical description of the multi-class RBF wLS-SVM model.

Algorithm 1 wLS-SVM with RBF kernel: training and multi-class classification

Require: Training data $\mathbf{X} \in \mathbb{R}^{n \times m}$, testing data $\mathbf{X}_{\text{test}} \in \mathbb{R}^{n_{\text{test}} \times m}$, label vector $\mathbf{y} \in \{-1, +1\}^n$, regularisation term $\lambda > 0$, RBF term $\gamma > 0$, stabilisation term $\delta > 0$, convergence threshold $\epsilon > 0$, number of classes C .

```

1:  $\mathbf{K} \leftarrow \exp^{\odot}(\gamma (2\mathbf{X}\mathbf{X}^T - \mathbf{q}\mathbf{1}^T - \mathbf{1}\mathbf{q}^T))$ , where  $q_i = \|p_i\|^2$ 
2: for each class  $c \in \{0, 1, \dots, C-1\}$  do
3:    $y_i^{(c)} \leftarrow \begin{cases} +1 & \text{if training label} = c \\ -1 & \text{otherwise} \end{cases}$ 
4:    $\mathbf{V}^{-1} \leftarrow \mathbf{I}_n$ 
5:   repeat
6:      $\begin{bmatrix} \mathbf{K} + \lambda\mathbf{V}^{-1} & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{\alpha}^{(c)} \\ w_0^{(c)} \end{bmatrix} = \begin{bmatrix} \mathbf{y}^{(c)} \\ 0 \end{bmatrix}$ 
7:      $\mathbf{e} \leftarrow \mathbf{K}\boldsymbol{\alpha}^{(c)} + w_0^{(c)}\mathbf{1} - \mathbf{y}^{(c)}$ 
8:      $\mathbf{V}_{\text{new}}^{-1} \leftarrow \text{diag}(e_1^2 + \delta, e_2^2 + \delta, \dots, e_n^2 + \delta)$ 
9:      $\text{change} \leftarrow \max_i \frac{|V_{ii,\text{new}}^{-1} - V_{ii}^{-1}|}{V_{ii}^{-1} + \epsilon}$   $\triangleright$  relative change from  $\mathbf{V}^{-1}$  to  $\mathbf{V}_{\text{new}}^{-1}$ 
10:     $\mathbf{V}^{-1} \leftarrow \mathbf{V}_{\text{new}}^{-1}$ 
11:   until change  $< \epsilon$ 
12:   Store  $\boldsymbol{\alpha}^{(c)}, w_0^{(c)}$ 
13:  $\mathbf{K}_{\text{test}} \leftarrow \exp^{\odot}(\gamma (2\mathbf{X}_{\text{test}}\mathbf{X}^T - \mathbf{q}_{\text{test}}\mathbf{1}^T - \mathbf{1}\mathbf{q}_{\text{test}}^T))$ 
14: for each class  $c \in \{0, 1, \dots, C-1\}$  do
15:    $\mathbf{f}^{(c)} \leftarrow \mathbf{K}_{\text{test}}\boldsymbol{\alpha}^{(c)} + w_0^{(c)}\mathbf{1}$ 
16:  $\hat{\mathbf{c}} \leftarrow \underset{c}{\text{argmax}} \begin{bmatrix} \mathbf{f}^{(0)} & \mathbf{f}^{(1)} & \dots & \mathbf{f}^{(C-1)} \end{bmatrix}$ 
return  $\hat{\mathbf{c}}$ 

```

This algorithm produces a vector, $\hat{\mathbf{c}}$, the element at index 0 is the predicted class for the first unseen testing point. The element at index 1 is the predicted class for the second unseen testing point, and so on.

A ‘change’ value is calculated at each iteration of the training loop, which measures the relative change from $\mathbf{V}^{-1}$ to $\mathbf{V}_{\text{new}}^{-1}$. This is to ensure that the training loop converges, and does not do unnecessary iterations after effective convergence has been reached.

An example output of the vector $\hat{\mathbf{c}}$ from [Algorithm 1](#) is shown below.

$$\hat{\mathbf{c}} = \begin{bmatrix} 7 & 2 & 1 & \cdots & 4 & 5 & 6 \end{bmatrix}^T. \quad (6.15)$$

In order to evaluate the performance of our model, we can compare $\hat{\mathbf{c}}$ with the vector of true classes for the testing data, which we call $\mathbf{y}_{\text{test}}$.

$$\mathbf{y}_{\text{test}} = \begin{bmatrix} 7 & 2 & 1 & \cdots & 4 & 5 & 6 \end{bmatrix}^T. \quad (6.16)$$

By comparing $\hat{\mathbf{c}}$ with $\mathbf{y}_{\text{test}}$, we count how many of the predicted classes are correct, and divide this by the total number of testing points. This gives us our accuracy score, which can be reported as a percentage.

Algorithm 2 Classification accuracy evaluation

Require: Predicted class vector $\hat{\mathbf{c}} \in \{0, 1, \dots, C-1\}^{n_{\text{test}}}$, true class vector $\mathbf{y}_{\text{test}} \in$

$$\{0, 1, \dots, C-1\}^{n_{\text{test}}}$$

$$1: z_i \leftarrow \begin{cases} 1 & \text{if } \hat{c}_i = y_{\text{test},i} \\ 0 & \text{otherwise} \end{cases}$$

$$2: \text{accuracy} \leftarrow \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} z_i$$

return {accuracy, accuracy $\times$ 100} $\triangleright$ accuracy as a decimal and as a percentage

We now have everything in place to train our multi-class RBF wLS-SVM model on the MNIST dataset, and evaluate its performance. In the next section, we will select various values for the number of training points n , as well as the regularisation term λ , and the RBF term γ . We will show how these changes affect the accuracy of our model, and we will also look at the model that performs best on the testing data.

6.3. Multi-class RBF wLS-SVM results for MNIST dataset

A good starting point for n is 1,000 training points. This is a very small subset of the training data, but we can be sure that our model will train in a reasonable amount of time. For λ and γ , we will pick the same starting value of 0.01. We previously found that $\lambda = 0.01$ is a good value for the regularisation term in our polynomial kernel version of the wLS-SVM model, so it is a natural choice.

Referring to [Algorithm 1](#), we will choose $\delta = 10^{-6}$ and $\epsilon = 10^{-10}$. We will try to keep these values fixed across all different combination of n , λ , and γ , since they are only used to ensure stability and convergence of our training loop. Making δ much smaller than 10^{-6} could make our weights in $\mathbf{V}^{-1}$ very large, which could lead to numerical instability. Changing ϵ could lead to early convergence (if ϵ is too large) or unnecessarily long training times (if ϵ is too small).

	n	λ	γ	accuracy % to 3 <i>d.p.</i>
Model 1	1,000	0.01	0.01	93.070%

Table 6.2: Multi-class RBF wLS-SVM model, using 1,000 training points.

This is a good result, considering we are only using $\sim 1.667\%$ of the training data. The training time for this model (on the machine used for this report) was approximately 0.5 seconds. We can, therefore, adjust λ and γ extensively, and train many different models for these 1,000 training points.

To keep this comparison fair, not only will we keep n fixed at 1,000, but we will use the exact same training points by utilising NumPy's ability to set a random seed. The random seed as a value we choose, and it ensures that the same random numbers are generated each time we run our code. For all of the models we train, for every value of n , we will set the random seed to 3. This is an arbitrary choice, and similar results would be obtained with any other random seed value.

Below is a table showing 6 different combinations of λ and γ , with n fixed at 1,000 training points.

	n	λ	γ	accuracy %
Model 2	1,000	0.01	0.1	86.63%
Model 3	1,000	0.01	0.001	89.77%
Model 4	1,000	0.01	0.021	93.39%
Model 5	1,000	1	0.01	90.74%
Model 6	1,000	0.000001	0.01	93.07%
Model 7	1,000	0.0001	0.0206	93.41%

Table 6.3: Multi-class RBF wLS-SVM model, using 1,000 training points.

After comparing the accuracy of these different pairs of λ and γ for the same 1,000 randomly selected training points, we can see that Model 7 performs the best. In order to ensure computational stability, we have to be careful when selecting λ and γ , which is why our best model only goes to 4 decimal places for λ and γ . Better results could possibly be obtained by refining λ and γ further, but we run the risk of introducing numerical instability.

Notice, in [Table 6.3.](#), that dramatically changing λ has a much smaller impact on the accuracy than even a small change in γ . This is because γ is used in $\exp(-\gamma\|p_i - p_j\|^2)$, where $\|p_i - p_j\|^2$ values can be very large. This means that small changes in γ can have large impacts on the value of $\exp(-\gamma\|p_i - p_j\|^2)$. Essentially, if γ is much larger than the optimal value, $\exp(-\gamma\|p_i - p_j\|^2) \approx 0$ for most pairs of points, meaning $\mathbf{K}$ becomes effectively diagonal, and the model is not good at capturing the relationships between the data points. The opposite is true if γ is much smaller than the optimal value, where it loses all discriminative power, $\exp(-\gamma\|p_i - p_j\|^2) \approx 1$ for most pairs of points, and our model becomes too simple. The same is not true for λ , since λ is used in $\mathbf{K} + \lambda\mathbf{V}^{-1}$, where much of the impact of λ can be absorbed by $\mathbf{V}^{-1}$, which is updated at each iteration. When λ becomes *too* large, it can make our model so simple that $\mathbf{V}^{-1}$ cannot compensate for it, and accuracy drops. Similarly, λ can become *too* small, to the point that $\mathbf{V}^{-1}$ cannot absorb its impact. Small λ values can also lead to numerical instability, so we must remain vigilant, even when accuracy remains high.

Due to the severe computational cost of increasing n , we are not able to train as many separate models for different λ and γ values, as we did for $n = 1,000$. Instead, we must use the best λ and γ values that we have: $\lambda = 0.0001$ and $\gamma = 0.0206$, and train a small number of higher- n models, with tweaks to these parameters. Below is a table of results showing the best model we were able to find for different values of n .

	n	λ	γ	accuracy %	training time
Model 8	2,000	0.0001	0.0190	94.69%	~ 2 s
Model 9	5,000	0.0001	0.0185	96.84%	~ 14 s
Model 10	7,500	0.0001	0.0185	97.28%	~ 35 s
Model 11	10,000	0.0001	0.0185	97.67%	~ 59 s
Model 12	15,000	0.0001	0.0185	98.00%	~ 168 s
Model 13	30,000	0.0001	0.0185	98.40%	~ 970 s
Model 14	60,000	0.0001	0.0185	98.76%	~ 5470 s

Table 6.4: Multi-class RBF wLS-SVM model, using higher values of n .

Due to the computing resources used for this report, our training time does not reflect the theoretical time complexity of our model, which is $\mathcal{O}(n^3)$ [11], and is instead much faster for smaller values of n . This is due to the machine utilising multiple CPU cores in parallel, allowing our model to drop closer to $\mathcal{O}(n^2)$ time complexity for smaller values of n . As n increases, and our model utilises all available CPU cores, the training time grows and approaches the theoretical $\mathcal{O}(n^3)$ time complexity.

We can see that, as we increase n , the accuracy of our model also increases, which is what we would expect. However, it is clear that larger values of n lead to much longer training times. Luckily, the training time is a one-off cost, and once we have trained our model, we can harvest α and w_0 , and store them. We then use these to perform predictions on unseen data points, as per Algorithm 1, without needing to retrain our model.

It is also evident that the accuracy improvements from increasing n are diminishing. Going from 2,000 to 5,000 training points (a 150% increase) gives us a 2.15% increase in accuracy, while going from 15,000 to 60,000 training points (a 300% increase) only gives us an accuracy increase of 0.76%. The code to generate the results in [Table 6.4](#) is available in the GitHub repository for this project, specifically in the [Appendices](#) under [Code 5](#), [Code 6](#), and [Code 7](#).

Both of these observations (dramatic increase in training time, and diminishing returns in accuracy) are expected for the kernel-based, SVM family of models. The increase in training time is because computation for SVMs relies on inverting an $n \times n$ matrix via LU decomposition, which has time complexity of $\mathcal{O}(n^3)$ [11]. The diminishing returns in accuracy are due to the fact that SVMs are not feature-extracting models like some deep learning models are, for example: convolutional neural networks (CNNs) [4].

Feature extracting models, in a sense, learn what **types** of features are found in the data [4]. They are able to learn that a certain combination of pixels in an image are an edge, or a corner, or a curve, and when they see this combination of pixels in a new image, they can identify it. This means that, if they have learned about the existence of enough different types of features, they can generalise well to unseen data points.

This is why SVMs are good for small datasets, and deep learning models are better for large datasets. [40] When the dataset is small, a deep learning model can't learn enough about the types of features in the data, and it can't be as sure what a new collection of pixels represents. On the other hand, SVMs don't rely on learning features, and just quantify the similarity between data points. If a new data point is more similar to one class than all the others, that is how SVMs will classify it, even if a deep learning model could pick up on a type of feature that makes the new data point more similar to a different class. wLS-SVMs just improve on this similarity-based classification, and give us a more robust model, bringing us closer to the performance of deep learning models on large datasets.

Conclusion

In this report, the core objective was to explore *how* we can derive least-squares support vector machines (LS-SVMs) all the way from linear regression. We then wanted to apply weightings to our LS-SVM, and see how the model performed on a substantial real-world dataset, the MNIST dataset.

Our wLS-SVM model was able to be derived successfully, and we were able to seamlessly integrate the concept of the radial basis function (RBF) kernel into our model, which allowed us to find infinite-degree relationships in our data. In the wider landscape of support vector machines, the RBF kernel is a highly used kernel, and is often the default choice for many mathematicians and computer scientists who want to use SVMs [41]. This is because the RBF kernel is very flexible, and as we have discussed in [Ch.6, Sec.1](#), it is highly tunable, allowing us to create higher and lower complexity models while always considering relationships in the data that are of infinite degree.

The application of our wLS-SVM model to the MNIST dataset was successful, and it showed that our model is able to achieve extremely high accuracies on the MNIST dataset. There are many other formal research papers that have applied different machine learning models to the MNIST dataset. Common types of models that are tested are traditional support vector machines (SVMs), convolutional neural networks (CNNs), and K-nearest neighbours (KNNs), just to name a few.

Common results from the literature show that CNNs tend to be the best performing models on the MNIST dataset, with accuracies consistently around or above 99% [38] when trained on the full 60,000 point training set. This is because of the way that CNNs are designed, which allows them to generalise extremely well for unseen data, when exposed to large training sets [42]. Areas you would expect to find CNNs for classification would be large-scale image recognition, and video analysis [42, 43]. However, SVMs (including our wLS-SVM model we have derived) are well-suited and widely used machine learning algorithms for small datasets, such as clinical diagnosis (where cohort sizes can be small, and data is often private) and rare disease research [40, 44]. Clinical fields *can* be both data-rich and data-poor, depending on how common the disease is, how much data is collected (or possible to collect), and how much data is shared. ML models that can work well on small datasets may, therefore, be very important for the future of clinical research.

Below is a table comparing the performance of our wLS-SVM model to some other ML models that have been exposed to similar sized (or larger) training sets of the MNIST dataset. The table is not exhaustive, but it gives a good indication of how our wLS-SVM model can perform on small datasets.

Model	Training Set Size	Test Accuracy
wLS-SVM (this report)	2,000	94.69%
wLS-SVM (this report)	5,000	96.84%
multi-class linear regression [37]	2,000	$\approx 83.00\%$
K-nearest neighbours (KNN) [37]	2,000	$\approx 90.00\%$
linear SVM [37]	2,000	$\approx 90.00\%$
polynomial SVM [37]	2,000	$\approx 92.50\%$
RBF SVM [37]	2,000	$\approx 94.00\%$
RBF SVM [37]	5,000	$\approx 96.00\%$
convolutional neural network (CNN) [38]	15,000	80.73%
CNN [38]	30,000	86.73%
CNN [38]	60,000	98.32%

Table 7.1: Model comparison on different training set sizes of the MNIST dataset.

This table shows us that the wLS-SVM model we have derived is not only able to achieve high accuracies on the MNIST dataset with small training set sizes, but it also outperforms some models that have been trained on the full 60,000 point training set. An important comparison to draw is how our 2,000-point wLS-SVM model performs considerably better than a 30,000-point CNN model, which is one of the most highly regarded models for image classification [42]. Our model could also, in theory, achieve even higher accuracies if the small training sets were chosen more strategically or preprocessed more carefully, for example, LeCun et al. [4] demonstrated that deslanting data prior to training can decrease error rate. This could allow our model to capture more of the variance in the data, and therefore generalise better to unseen digits, without needing to be exposed to a larger training set.

It must be noted that the wLS-SVM is not the all-around best performing model across the board. As the number of training points increases, the effect of holding all the data points in an $n \times n$ matrix becomes increasingly significant, and the model becomes exponentially slower [11]. On top of this, our wLS-SVM model is not able to keep up with the performance of specialised versions of CNNs, which exceed 99.70% accuracy on the full training set, in some reports [45]. When training points reach the tens, or hundreds, or thousands, other models are more suitable [42, 46].

In conclusion, we can now answer the question:

When are wLS-SVMs the best choice of model?

From reading this report, it is clear that wLS-SVMs are one of the best choices of model for small datasets. This is due to their ability to capture complex relationships, without needing large volumes of data, while remaining mathematically simple and computationally efficient. LS-SVMs and wLS-SVMs also remain an active area of research, with mathematicians and computer scientists working on improving the accuracy and computational efficiency of the model. For example, the ‘Adaptive Boost’ LS-SVM, or AdaBoost LS-SVM, is an algorithm that also adds weightings to the LS-SVM, using a separate approach from wLS-SVMs, with great success [47]. The wLS-SVM continues to be a powerful, flexible, and interpretable model, which serves as an important part of the rich history of support vector machines... one of the most well-known and widely used machine learning algorithms of all time.



Appendices

Below is the GitHub link to the full code for our kernel ridge regression model, which was used to calculate the predictions in [Ch.3, Sec.3](#). The code stores our data in NumPy arrays, and uses NumPy's built-in linear algebra solver: `numpy.linalg.solve()`, which solves via *LU* decomposition, to find α . These α values are then used to find $\hat{y}$ values for a selection of new data points.

Code 1 Link to full kernel ridge regression Python code.

[https://github.com/weatherill-james/BSc-Project-code/blob/
main/code/kernel_ridge_regression.py](https://github.com/weatherill-james/BSc-Project-code/blob/main/code/kernel_ridge_regression.py)

Table A.1: Positive Class

	x_1	x_2	y	$f(p_i)$ to 3 d.p.	$sign(f(p_i))$
p_1	0	0	+1	1.522	+1
p_2	0.5	0	+1	0.899	+1
p_3	0.35	0.35	+1	0.911	+1
p_4	0	0.5	+1	0.899	+1
p_5	-0.35	0.35	+1	0.911	+1
p_6	-0.5	0	+1	0.899	+1
p_7	-0.35	-0.35	+1	0.911	+1
p_8	0	-0.5	+1	0.899	+1
p_9	0.35	-0.35	+1	0.911	+1

Table A.2: Negative Class

	x_1	x_2	y	$f(p_i)$ to 3 d.p.	$sign(f(p_i))$
p_{10}	1	0	-1	-0.969	-1
p_{11}	0.87	0.5	-1	-0.986	-1
p_{12}	0.5	0.87	-1	-0.986	-1
p_{13}	0	1	-1	-0.969	-1
p_{14}	-0.5	0.87	-1	-0.986	-1
p_{15}	-0.87	0.5	-1	-0.986	-1
p_{16}	-1	0	-1	-0.969	-1
p_{17}	-0.87	-0.5	-1	-0.986	-1
p_{18}	-0.5	-0.87	-1	-0.986	-1
p_{19}	0	-1	-1	-0.969	-1
p_{20}	0.5	-0.87	-1	-0.986	-1
p_{21}	0.87	-0.5	-1	-0.986	-1

Figure A.1: LS-SVM bullseye training data with predictions

Figure A.1 shows two tables: Table A.1 and Table A.2, which reflect the actual classes, raw predicted values, and predicted classes for the training points in the bullseye dataset, using an LS-SVM model. The results show that the LS-SVM correctly classifies all the training points, which is a good sign, and gives us confidence that the model can correctly classify unseen data points, as we can see in Table 5.3.

Below is the GitHub link to the full code for our least-squares support vector machine model, which was used to calculate the predicted classes in [Ch.5, Sec.2](#). Similar to the code for kernel ridge regression, This code stores both our training data and new data in NumPy arrays, and this time uses the sign of the predicted value to determine the predicted class. The α and w_0 values *could* then be harvested and stored so that we could make predictions for any new data point, without having to retrain the model. We don't do this in this code, since the training time is so fast, but it is something that (if necessary) could easily be implemented.

Code 2 Link to full LS-SVM Python code for the bullseye dataset.

[https://github.com/weatherill-james/BSc-Project-code/blob/
main/code/ls_svm_bullseye.py](https://github.com/weatherill-james/BSc-Project-code/blob/main/code/ls_svm_bullseye.py)

Table A.3: LS-SVM predictions

	x_1	x_2	y	$f(p_i)$	$sign$
p_1	-2.5	1.5	+1	0.401	+1
p_2	-3.5	1.5	+1	0.797	+1
p_3	-3.5	0.5	+1	0.803	+1
p_4	-2.5	0.5	+1	0.407	+1
p_5	-3	1	+1	0.797	+1
p_6	-6	2	+1	1.486	+1
p_7	-5	1	+1	1.733	+1
p_8	-4	0	+1	0.419	+1
p_9	-4	2	+1	0.406	+1
p_{10}	-6	0	+1	1.500	+1
p_{11}	5.5	1.5	+1	-0.155	-1
p_{12}	5	0.5	+1	-0.252	-1
p_{13}	6	1	+1	0.170	+1
p_{14}	1	1	?	-0.206	-1
p_{15}	0.5	1.5	?	-0.351	-1
p_{16}	-0.5	1.5	?	-0.172	-1
p_{17}	-1	1	?	0.151	+1
p_{18}	-0.5	0.5	?	-0.167	-1
p_{19}	0.5	0.5	?	-0.345	-1
p_{20}	2.5	1.5	-1	-0.490	-1
p_{21}	3.5	1.5	-1	-0.451	-1
p_{22}	3.5	0.5	-1	-0.446	-1
p_{23}	2.5	0.5	-1	-0.485	-1
p_{24}	3	1	-1	-0.273	-1
p_{25}	6	2	-1	-0.642	-1
p_{26}	5	1	-1	-0.050	-1
p_{27}	4	0	-1	-1.009	-1
p_{28}	4	2	-1	-1.019	-1
p_{29}	6	0	-1	-0.651	-1

Table A.4: wLS-SVM predictions

	x_1	x_2	y	$f(p_i)$	$sign$
p_1	-2.5	1.5	+1	0.667	+1
p_2	-3.5	1.5	+1	1.000	+1
p_3	-3.5	0.5	+1	1.000	+1
p_4	-2.5	0.5	+1	0.667	+1
p_5	-3	1	+1	1.000	+1
p_6	-6	2	+1	1.500	+1
p_7	-5	1	+1	1.750	+1
p_8	-4	0	+1	0.667	+1
p_9	-4	2	+1	0.667	+1
p_{10}	-6	0	+1	1.500	+1
p_{11}	5.5	1.5	+1	-0.500	-1
p_{12}	5	0.5	+1	-0.505	-1
p_{13}	6	1	+1	-0.313	-1
p_{14}	1	1	?	-4.959×10^{-5}	-1
p_{15}	0.5	1.5	?	-8.336×10^{-2}	-1
p_{16}	-0.5	1.5	?	1.250×10^{-1}	+1
p_{17}	-1	1	?	4.166×10^{-1}	+1
p_{18}	-0.5	0.5	?	1.250×10^{-1}	+1
p_{19}	0.5	0.5	?	-8.336×10^{-2}	-1
p_{20}	2.5	1.5	-1	-0.375	-1
p_{21}	3.5	1.5	-1	-0.458	-1
p_{22}	3.5	0.5	-1	-0.458	-1
p_{23}	2.5	0.5	-1	-0.375	-1
p_{24}	3	1	-1	-0.250	-1
p_{25}	6	2	-1	-1.000	-1
p_{26}	5	1	-1	-0.333	-1
p_{27}	4	0	-1	-1.000	-1
p_{28}	4	2	-1	-1.000	-1
p_{29}	6	0	-1	-1.000	-1

Figure A.2: LS-SVM and wLS-SVM outlier dataset predictions where $f(p_i)$ is to 3 *d.p.*, and $sign$ is the predicted class. $\lambda = 0.01$ and $d = 2$ for both models. $\delta = 1 \times 10^{-6}$

[Table A.3](#) and [Table A.4](#) in [Figure A.2](#) show the predicted values for both the regular least-squares support vector machine and the weighted least-squares support vector machine on the outlier dataset. Both tables include the training data points and the new data points. We can see that the wLS-SVM model gives predicted classes that are more in line with the class we would expect, where outliers are misclassified, and all other points are correctly classified. We can visually see how the LS-SVM and wLS-SVM classify the new data points in [Figure A.3](#) and [Figure A.4](#) below.

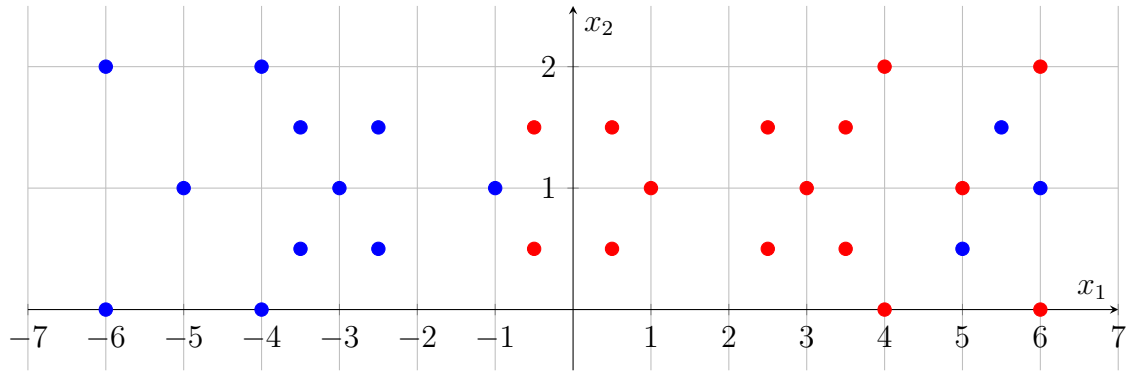


Figure A.3: Outlier dataset: LS-SVM Unknown Point Classification.

Blue: +1 class; Red: -1 class

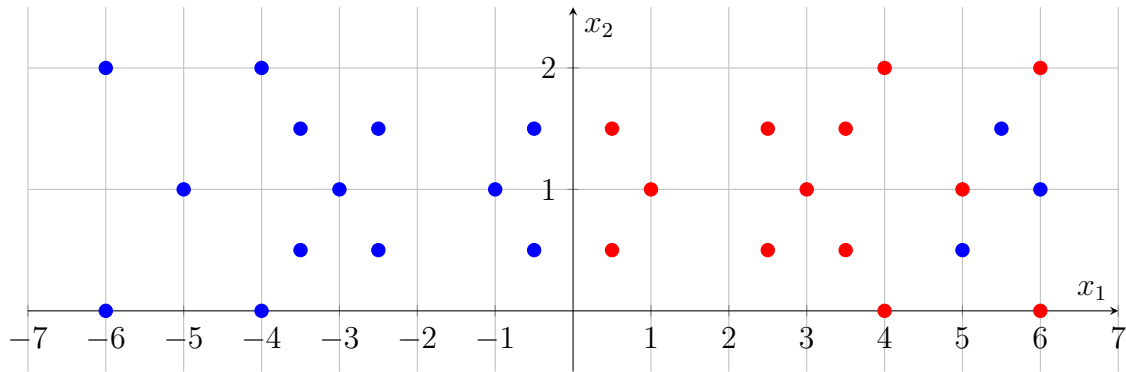


Figure A.4: Outlier dataset: wLS-SVM Unknown Point Classification.

Blue: +1 class; Red: -1 class

Figure A.3 and Figure A.4 show the predicted classes, visually, for the new data points in the outlier dataset. The original classes of the training data are kept the same as in Figure 5.4, for both plots, so the only differences are the classifications for the six new data points. We can see that the LS-SVM model classifies only the furthest left new point as +1, and the rest as -1, which implies the model is negatively affected by the outliers. The wLS-SVM model, however, classifies the three leftmost new points as +1, and the three rightmost new points as -1. This is what we would expect, and shows the wLS-SVM model has the ability to suppress the influence of outliers, and give more accurate predictions for unseen data points.

Table A.5: wLS-SVM final weightings in $\mathbf{V}$

	x_1	x_2	y	$f(p_i)$	$sign$	final weight to 3 <i>d.p.</i>
p_1	-2.5	1.5	+1	0.401	+1	9.000
p_2	-3.5	1.5	+1	0.797	+1	999940.999
p_3	-3.5	0.5	+1	0.803	+1	999940.352
p_4	-2.5	0.5	+1	0.407	+1	9.000
p_5	-3	1	+1	0.797	+1	999785.328
p_6	-6	2	+1	1.486	+1	3.999
p_7	-5	1	+1	1.733	+1	1.778
p_8	-4	0	+1	0.419	+1	9.003
p_9	-4	2	+1	0.406	+1	9.003
p_{10}	-6	0	+1	1.500	+1	3.999
p_{11}	5.5	1.5	+1	-0.155	-1	0.444
p_{12}	5	0.5	+1	-0.252	-1	0.441
p_{13}	6	1	+1	0.170	+1	0.580
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	
p_{20}	2.5	1.5	-1	-0.490	-1	2.560
p_{21}	3.5	1.5	-1	-0.451	-1	3.409
p_{22}	3.5	0.5	-1	-0.446	-1	3.409
p_{23}	2.5	0.5	-1	-0.485	-1	2.560
p_{24}	3	1	-1	-0.273	-1	1.778
p_{25}	6	2	-1	-0.642	-1	999990.383
p_{26}	5	1	-1	-0.050	-1	2.250
p_{27}	4	0	-1	-1.009	-1	999995.010
p_{28}	4	2	-1	-1.019	-1	999995.024
p_{29}	6	0	-1	-0.651	-1	999990.497

Table A.5 shows the all of the training points used for our outlier dataset, along with their wLS-SVM raw predicted values, and predicted classes. It also shows the final weighting received for each point in the $\mathbf{V}$ matrix after the final re-weighting iteration. Only original training points are shown, since our model is not trained on new data points, and thus they do not receive any weightings. Points that have been deemed outliers by our model, are given very low weightings, meaning they have very little influence our decision boundary. Points that are not deemed outliers, are given much higher weightings, and their positions influence the decision boundary much more.

Below are the GitHub links to the full code for our regular least-squares support vector machine, as well as our weighted least-squares support vector machine model. These were used to calculate the predicted classes for the dataset created and shown in [Ch.5](#), with results shown in [Figure A.2](#). [Code 3](#) shows the implementation of the regular LS-SVM, where the code is a natural extension of [Code 2](#). [Code 4](#) is a more computationally efficient approach of our previous LS-SVM implementations, where we utilise more of NumPy's built-in functions. We do this here due to the computational load that is added by introducing the weighting matrix, $\mathbf{V}$. This efficiency gain is also essential for more complex data and/or larger datasets, such as the MNIST dataset.

Code 3 Link to full LS-SVM Python code for the outlier dataset.

[https://github.com/weatherill-james/BSc-Project-code/blob/
main/code/ls_svm_outlier_data.py](https://github.com/weatherill-james/BSc-Project-code/blob/main/code/ls_svm_outlier_data.py)

Code 4 Link to full wLS-SVM Python code for the outlier dataset.

[https://github.com/weatherill-james/BSc-Project-code/blob/
main/code/w_ls_svm_outlier_data.py](https://github.com/weatherill-james/BSc-Project-code/blob/main/code/w_ls_svm_outlier_data.py)

Below are the GitHub links to the full code for the multi-class RBF wLS-SVM model, which each serve a different purpose. Since the MNIST dataset is so large (60,000 training points), [Code 5](#) links to the code file used to break the MNIST dataset into smaller subsets, where each point is randomly chosen and the subset contains the same number of points from each class. [Code 6](#) is the link to the file used to train the multi-class RBF wLS-SVM model on the full MNIST dataset. Since the MNIST dataset does not contain equal numbers of points from each class, the code in [Code 5](#) does not work for the full dataset. Beyond this, the method used to train the model remains identical between [Code 5](#) and [Code 6](#). [Code 7](#) is the link to the file used to make predictions on the MNIST dataset via pre-trained models that we created. 8 pre-trained models are included in the GitHub repository, which were trained on 1,000, 2,000, 5,000, 7,500, 10,000, 15,000, 30,000, and 60,000 training points. Results for these models are shown in [Ch.6, Sec.3](#).

All three files can be tweaked to train the model on new subset sizes, or to use a different seed, or to find results from different pre-trained models. The parameters γ , λ , δ , and ϵ can also be changed in the code files.

Code 5 Link to full multi-class RBF wLS-SVM Python code for subsets of the MNIST dataset.

https://github.com/weatherill-james/BSc-Project-code/blob/main/code/w_ls_svm_RBF_MNIST.py

Code 6 Link to full multi-class RBF wLS-SVM Python code for the full MNIST dataset.

https://github.com/weatherill-james/BSc-Project-code/blob/main/code/w_ls_svm_RBF_MNIST_full.py

Code 7 Link to full multi-class RBF wLS-SVM Python code for pre-trained models on the MNIST dataset.

https://github.com/weatherill-james/BSc-Project-code/blob/main/code/w_ls_svm_RBF_MNIST_no_training.py

References

- [1] P. Linardatos, V. Papastefanopoulos, and S. Kotsiantis, *Explainable AI: A Review of Machine Learning Interpretability Methods*, [Entropy](#) **23** (2020) 18.
- [2] K. Tomita, T. Nishida, Y. Kitaguchi, K. Kitazawa, and M. Miyake, *Image Recognition Performance of GPT-4V(ision) and GPT-4o in Ophthalmology: Use of Images in Clinical Questions*, [Clinical Ophthalmology Volume](#) **19** (2025) 1557.
- [3] OpenAI. *ChatGPT — Release Notes*, .
- [4] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, *Gradient-based Learning Applied to Document Recognition*, *Proceedings of the IEEE* **86** (1998) 2278.
- [5] H. Drucker, D. Wu, and V. Vapnik, *Support vector machines for spam categorization*, [IEEE Transactions on Neural Networks](#) **10** (1999) 1048.
- [6] R. Guido, S. Ferrisi, D. Lofaro, and D. Conforti, *An Overview on the Advancements of Support Vector Machine Models in Healthcare Applications: A Review*, [Information](#) **15** (2024) 235.
- [7] G. James, D. Witten, T. Hastie, and R. Tibshirani. *An Introduction to Statistical Learning with Applications in R Second Edition*, 2021.
- [8] C. Bishop, *Pattern Recognition and Machine Learning*, [Journal of Electronic Imaging](#) **16** (2007) 049901.
- [9] A. N. Tikhonov and V. I. Arsenin, *Solutions of Ill-posed Problems*, Winston Publishing, 1977.
- [10] A. E. Hoerl and R. W. Kennard, *Ridge Regression: Biased Estimation for Nonorthogonal Problems*, [Technometrics](#) **12** (1970) 55.
- [11] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*, Mit Press, 2009.
- [12] C. F. Gauss, *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*, Friedrich Perthes and I.H. Besser, 1809.
- [13] J. Mercer, *Functions of positive and negative type, and their connection with the theory of integral equations*, [Proceedings of the Royal Society of London Series A Containing Papers of a Mathematical and Physical Character](#) **83** (1909) 69.
- [14] D. Hilbert, *Grundzüge einer allgemeinen Theorie der linearen Integralrechnungen. (Zweite Mitteilung)*, *Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse* **1904** (1904) 213.

-
- [15] M. Aizerman, E. Braverman, and L. Rozonoer, *Theoretical foundations of the potential function method in pattern recognition learning*, Automation and Remote Control **25** (1964) 821.
 - [16] V. N. Vapnik, *Statistical Learning Theory*, Wiley-Interscience, 09 1998.
 - [17] R. A. FISHER, *THE USE OF MULTIPLE MEASUREMENTS IN TAXONOMIC PROBLEMS*, [Annals of Eugenics](#) **7** (1936) 179.
 - [18] F. Rosenblatt, *The perceptron: A probabilistic model for information storage and organization in the brain.*, [Psychological Review](#) **65** (1958) 386.
 - [19] V. Vapnik, *Pattern recognition using generalized portrait method*, Automation and Remote Control **24** (1963) 774.
 - [20] A. J. Smola and B. Schölkopf, *A tutorial on support vector regression*, [Statistics and Computing](#) **14** (2004) 199.
 - [21] V. Vapnik, *Estimation of Dependences Based on Empirical Data*, Springer New York, 2006.
 - [22] C. J. Burges, *A Tutorial on Support Vector Machines for Pattern Recognition*, [Data Mining and Knowledge Discovery](#) **2** (1998) 121.
 - [23] TheFranklinInstitute. *Vladimir Vapnik*, 09 2014.
 - [24] I. Guyon. *Data Mining History: The Invention of Support Vector Machines*, *KDNuggets*, 2016.
 - [25] B. E. Boser, I. M. Guyon, and V. N. Vapnik, *A training algorithm for optimal margin classifiers*, [Proceedings of the fifth annual workshop on Computational learning theory - COLT '92\(1992\)](#).
 - [26] T. Poggio, *On optimal nonlinear associative recall*, [Biological Cybernetics](#) **19** (1975) 201.
 - [27] A. J. Smola, B. Schölkopf, and K.-R. Müller, *The connection between regularization operators and support vector kernels*, [Neural Networks](#) **11** (1998) 637.
 - [28] C. Yadav. *Cold Case: The Lost MNIST Digits*, 2019.
 - [29] C. Cortes and V. Vapnik, *Support-vector networks*, [Machine Learning](#) **20** (1995) 273.
 - [30] J. Suykens and J. Vandewalle, *Least Squares Support Vector Machine Classifiers*, [Neural Processing Letters](#) **9** (1999) 293.

- [31] J. Suykens, J. De Brabanter, L. Lukas, and J. Vandewalle, *Weighted least squares support vector machines: robustness and sparse approximation*, [Neurocomputing](#) **48** (2002) 85.
- [32] J. Suykens, T. Van Gestel, J. De Brabanter, B. De Moor, and J. Vandewalle, *Least squares support vector machines*, World Scientific, 2002.
- [33] A. D. Peterson, A. P. Ghosh, and R. Maitra, *Merging K -means with hierarchical clustering for identifying general-shaped groups*, [Stat](#) **7** (2018) e172.
- [34] G. H. Golub and C. F. Van, *Matrix computations*, The Johns Hopkins University Press, Cop, 2013.
- [35] H. D. Nguyen and G. J. McLachlan. *Iteratively-Reweighted Least-Squares Fitting of Support Vector Machines: A Majorization–Minimization Algorithm Approach*, 2017.
- [36] B. Schölkopf and A. J. Smola, *Learning with kernels : support vector machines, regularization, optimization, and beyond*, Mit Press, 2002.
- [37] P. M. Nguyen. *Machine Learning with Insufficient Data Amount*, 2016.
- [38] F. Chen, N. Chen, H. Mao, and H. Hu. *Assessing Four Neural Networks on Hand-written Digit Recognition Dataset (MNIST)*, 2018.
- [39] H. Chih-Wei and L. Chih-Jen, *A comparison of methods for multiclass support vector machines*, [IEEE Transactions on Neural Networks](#) **13** (2002) 415.
- [40] P. Wang, E. Fan, and P. Wang, *Comparative analysis of image classification algorithms based on traditional machine learning and deep learning*, [Pattern Recognition Letters](#) **141** (2021).
- [41] H. Chih-Wei, C. Chih-Chung, and L. Chih-Jen. *A Practical Guide to Support Vector Classification*, 2003.
- [42] A. Krizhevsky, I. Sutskever, and G. E. Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, *Communications of the ACM* **60** (2012) 84.
- [43] A. Karpathy, G. Toderici, S. Shetty, T. Leung, R. Sukthankar, and L. Fei-Fei. *Large-scale Video Classification with Convolutional Neural Networks*, 2014.
- [44] M. F. Akay, *Support vector machines combined with feature selection for breast cancer diagnosis*, [Expert Systems with Applications](#) **36** (2009) 3240.
- [45] D. Cireşan, U. Meier, and J. Schmidhuber, *Multi-column Deep Neural Networks for Image Classification*, [CoRR](#) **abs/1202.27** (2012).

- [46] D. Cireşan and J. Schmidhuber, *Multi-Column Deep Neural Networks for Offline Handwritten Chinese Character Classification*, [CoRR a bs/1309.02](#) (2013).
- [47] H. Al-Hadeethi, S. Abdulla, M. Diykh, R. C. Deo, and J. H. Green, *Adaptive boost LS-SVM classification approach for time-series signal classification in epileptic seizure diagnosis applications*, [Expert Systems with Applications](#) **161** (2020) 113676.

